

슬래시 명령어 완벽 가이드

1. 슬래시 명령어란?

슬래시 명령어는 Claude Code 세션 내부에서 Claude 를 제어하는 수단입니다. 모델 전환, 권한 관리, 컨텍스트 초기화, 워크플로우 실행 등을 빠르게 처리할 수 있습니다. /를 입력하면 사용 가능한 전체 명령어 목록이 나타나고, / 뒤에 문자를 입력하면 필터링됩니다. 명령어는 반드시 메시지의 맨 앞에서만 인식됩니다.

슬래시 명령어는 총 4 가지 종류가 있습니다:

/ 입력 시 나타나는 명령어들

- ● Built-in 명령어 → Claude Code 에 내장된 기능
- ● Bundled Skills → SKILL.md 방식의 내장 워크플로우
- ● Custom 명령어 → 직접 만든 .claude/skills/ 파일
- ● MCP 명령어 → 연결된 MCP 서버의 프롬프트

2. 세션 & 컨텍스트 관리 명령어

대화의 상태와 흐름을 제어하는 핵심 명령어들입니다.

명령어	설명	실전 팁
<code>/clear</code>	대화 기록 전체 삭제, 컨텍스트 초기화	완전히 다른 작업으로 전환할 때
<code>/compact</code> [지시]	이전 대화를 요약으로 압축해 컨텍스트 확보	작업은 계속하되 토큰이 부족할 때
<code>/branch</code> [이름]	현재 대화를 분기해 새 세션 생성	위험한 시도를 해볼 때. 별칭: <code>/fork</code>
<code>/resume</code> [세션]	이전 대화 세션 재개	어제 하던 작업 이어서 할 때. 별칭: <code>/continue</code>
<code>/rewind</code>	대화과 코드 변경을 이전 체크포인트로 되돌리기	Claude가 잘못된 방향으로 갔을 때. 별칭: <code>/checkpoint</code>
<code>/rename</code> [이름]	현재 세션 이름 지정	세션 목록에서 쉽게 찾기 위해

/clear 와 /compact 의 차이: /clear 는 대화 기록 전체를 삭제합니다. /compact 는 대화 기록을 압축된 요약으로 교체해 앞서 논의한 맥락은 유지하면서 토큰 예산을 확보합니다. 완전히 다른 작업으로 전환할 때는 /clear, 같은 작업을 계속하면서 한계에 가까워졌을 때는 /compact 를 쓰세요.

3. 설정 & 환경 관리 명령어

명령어	설명	비고
<code>/config</code>	테마, 모델, 출력 스타일 등 설정 UI 열기	별칭: <code>/settings</code>
<code>/model</code> [모델명]	AI 모델 전환	즉시 효과 적용, 좌우 화살표로 effort 조절
<code>/effort</code> [low medium high max auto]	모델 추론 깊이 설정	<code>max</code> 는 Opus 4.6 전용, 현재 세션만 적용
<code>/permissions</code>	도구 허용/요청/차단 규칙 관리	별칭: <code>/allowed-tools</code>
<code>/memory</code>	CLAUDE.md 메모리 파일 편집 및 auto-memory 관리	프로젝트 영속 기억 관리
<code>/init</code>	프로젝트에 CLAUDE.md 초기화	새 프로젝트 시작 시
<code>/hooks</code>	라이프사이클 훅 설정 확인	도구 실행 전후 자동 명령 등록
<code>/sandbox</code>	샌드박스 모드 토글	격리된 환경에서 안전하게 실행
<code>/color</code> [색상]	현재 세션 프롬프트 바 색상 변경	여러 세션 구분할 때 유용

4. 대규모 작업 & 자동화 명령어 (Bundled Skills)

Bundled Skills 로 표시된 항목들은 직접 작성한 Skills 와 동일한 방식으로 동작합니다. Claude 에게 전달되는 프롬프트이며, 관련 상황에서 Claude 가 자동으로 호출할 수도 있습니다.

`/batch` <지시사항> ★ 핵심

대규모 코드베이스 변경을 병렬로 조율합니다. 코드베이스를 조사하고 작업을 5~30 개의 독립 단위로 분해해 계획을 제시합니다. 승인 후 각 단위마다 격리된 git worktree 에서 백그라운드 에이전트를 생성합니다. 각 에이전트는 해당 단위를 구현하고, 테스트를 실행하고, 풀 리퀘스트를 엽니다. git 저장소가 필요합니다.

```
bash

# 예시: React 마이그레이션 전체를 병렬로 처리
/batch migrate src/ from Solid to React

# 예시: 전체 코드베이스 타입 추가
/batch add TypeScript types to all JavaScript files in src/
```

/loop [간격] [프롬프트]

세션이 열려 있는 동안 프롬프트를 반복 실행합니다. 간격을 생략하면 Claude 가 반복 간 속도를 스스로 조절합니다. 프롬프트를 생략하면 .claude/loop.md 의 프롬프트를 실행하거나 자율 유지보수 검사를 실행합니다.

```
bash

# 5분마다 배포 상태 확인
/loop 5m check if the deploy finished

# 매시간 로그 모니터링
/loop 1h scan server logs for errors and alert me
```

/schedule [설명]

루틴을 생성, 업데이트, 목록 조회 또는 실행합니다. Claude 가 대화 방식으로 설정을 안내합니다.

```
bash

# 예약 작업 생성 대화 시작
/schedule daily code review at 9am
```

/autofix-pr [프롬프트]

현재 브랜치의 PR 을 감시하고 CI 가 실패하거나 리뷰어가 코멘트를 남기면 자동으로 수정을 푸시하는 웹 세션을 생성합니다. gh CLI 와 Claude Code on the web 접근이 필요합니다.

```
bash

# 모든 CI 실패와 리뷰 코멘트 자동 수정
/autofix-pr

# 특정 오류만 수정
/autofix-pr only fix lint and type errors
```

5. 개발 워크플로우 명령어

명령어	설명
/plan [설명]	플랜 모드 진입. 코드 작성 전 구현 계획 수립
/diff	미커밋 변경사항과 특별 diff 인터랙티브 뷰어
/security-review	현재 브랜치 변경사항 보안 취약점 분석
/debug [설명]	디버그 로깅 활성화 후 세션 로그 분석
/context	현재 컨텍스트 사용량 시각화 + 최적화 제안
/add-dir <경로>	현재 세션에 작업 디렉토리 추가

/plan 실전 활용

```
bash

# 구현 시작 전 먼저 계획 수립
/plan implement JWT authentication for the Express API

# Claude가 관련 파일 탐색 → 구현 계획 제시 → 승인 후 실행
```

/security-review 실전 활용


```
bash

# 현재 브랜치 변경사항의 보안 취약점 자동 분석
/security-review

# 결과: 인젝션, 인증 우회, 데이터 노출 등 위험도별 분류
```

6. 에이전트 & 확장 관리 명령어

명령어	설명
<code>/agents</code>	Subagent 설정 관리 (생성/편집/목록)
<code>/plugin</code>	Plugin 설치/제거/목록 관리
<code>/reload-plugins</code>	재시작 없이 변경된 플러그인 즉시 재로드
<code>/mcp</code>	MCP 서버 연결 및 OAuth 인증 관리

7. MCP 슬래시 명령어

MCP 서버를 연결하면 해당 서버의 프롬프트가 `/mcp__[서버명]__[프롬프트명]` 형식의 슬래시 명령어로 노출됩니다.

```
bash
```

```
# GitHub MCP 서버 연결 후:
```

```
/mcp__github__list_prs
```

```
/mcp__github__create_issue
```

```
# Slack MCP 서버 연결 후:
```

```
/mcp__slack__send_message
```

```
/mcp__slack__search_messages
```

```
# Notion MCP 서버 연결 후:
```

```
/mcp__notion__create_page
```

```
/mcp__notion__search
```

8. 유틸리티 & 기타 명령어

명령어	설명
<code>/copy [N]</code>	마지막 응답 클립보드 복사. N번째 이전 응답 지정 가능
<code>/export [파일명]</code>	현재 대화를 텍스트 파일로 내보내기
<code>/btw <질문></code>	대화에 추가하지 않고 빠른 사이드 질문
<code>/cost</code>	토큰 사용량 통계 확인
<code>/insights</code>	세션 분석 리포트 생성 (작업 패턴, 마찰 지점 등)
<code>/release-notes</code>	버전별 변경 로그 확인
<code>/powerup</code>	애니메이션 데모로 Claude Code 기능 배우기
<code>/doctor</code>	설치 환경 진단 및 이슈 자동 수정
<code>/feedback</code>	피드백/버그 리포트 제출. 별칭: <code>/bug</code>
<code>/desktop</code>	현재 세션을 Claude Code Desktop 앱으로 이어서 실행
<code>/remote-control</code>	현재 세션을 claude.ai에서 원격 제어 가능하게 설정
<code>/install-github-app</code>	Claude GitHub Actions 앱 설정
<code>/install-slack-app</code>	Claude Slack 앱 설치

9. 커스텀 슬래시 명령어 만들기

기본 구조

커스텀 명령어는 `.claude/skills/` 디렉토리에 저장된 마크다운 파일입니다. 슬래시 명령어로 호출할 수 있는 재사용 가능한 프롬프트를 정의합니다.

저장 위치에 따른 범위:

<code>.claude/skills/</code>	→ 프로젝트 전용 (<code>git</code> 으로 팀 공유)
<code>~/.claude/skills/</code>	→ 개인 전용 (모든 프로젝트에서 사용)

YAML 프론트매터 옵션

커스텀 명령어는 YAML 프론트매터로 다양한 설정을 지원합니다. `allowed-tools`로 사용 가능한 도구를 제한하고, `argument-hint`로 인수 힌트를 표시하며, `model`로 특정 모델을 지정하고, `context: fork`로 현재 대화를 분기해 실행할 수 있습니다.

```
markdown
---
name: commit
description: 컨텍스트를 담은 git 커밋 생성
allowed-tools: Bash(git add:*), Bash(git status:*), Bash(git commit:*)
argument-hint: "[커밋 메시지]"
model: claude-haiku-4-5
context: fork
---

## 컨텍스트
- 현재 git 상태: !`git status`
- 변경 파일: !`git diff --name-only`

## 작업
1. 변경사항을 분석해 Conventional Commits 형식으로 커밋 메시지 작성
2. `git add -A` 실행
3. 커밋 메시지로 커밋
```

\$ARGUMENTS 변수 활용

`/fix-issue 123 high`처럼 입력하면 `$ARGUMENTS`에 "123 high"가 전달됩니다.

markdown

name: pr-review

description: PR 코드 리뷰

allowed-tools: Read, Grep, Bash(git diff:*)

변경된 파일

!`git diff --name-only HEAD~1`

상세 변경사항

!`git diff HEAD~1`

리뷰 기준

1. 로직 오류 및 엣지 케이스
2. 보안 취약점 (인젝션, XSS, 인증 우회)
3. 성능 이슈
4. 테스트 커버리지

10. Plugin 슬래시 명령어

Plugin 이 설치되면 해당 Plugin 의 명령어가 `/플러그인명:명령어명` 형식으로 노출됩니다.

```
bash

# Sales Plugin 명령어
/sales:call-prep Acme Corp
/sales:follow-up

# Legal Plugin 명령어
/legal:review-contract
/legal:triage-nda

# Engineering Plugin 명령어
/engineering:code-review
/engineering:security-scan

# Data Plugin 명령어
/data:write-query 월별 매출 추이
/data:dashboard
```

11. 명령어 우선순위

같은 이름의 Skill 과 명령어가 공존하면 Skill 이 우선합니다. 예를 들어 `.claude/commands/review.md` 와 `.claude/skills/review/SKILL.md` 가 모두 있으면 Skill 버전이 사용됩니다.

우선순위 (높음 → 낮음):

1. 커스텀 Skills (`.claude/skills/`)
2. 커스텀 Commands (`.claude/commands/`) ← 레거시, 하위 호환
3. Plugin 명령어
4. MCP 프롬프트 명령어
5. Bundled Skills (내장 스킬)
6. Built-in 명령어 (내장)

12. 자주 쓰는 명령어 cheatsheet

가장 활발하게 Claude Code 를 사용하는 사람들이 매일 쓰는 핵심 명령어들입니다.

```
bash

# ── 매일 쓰는 5가지 ──
/clear          # 새 작업 시작 전 컨텍스트 정리
/compact        # 작업 중 토큰 부족 시
/plan           # 큰 작업 전 계획 먼저
/rewind          # 잘못된 방향으로 갔을 때 되돌리기
/cost           # 사용량 확인

# ── 개발 워크플로우 ──
/security-review # PR 전 보안 점검
/batch          # 대규모 변경 병렬 처리
/autofix-pr     # CI 실패 자동 수정
/diff           # 변경사항 시각적 확인
/debug          # 문제 추적

# ── 설정 관리 ──
/model          # 모델 전환
/effort max     # 복잡한 작업에 최대 추론
/permissions    # 도구 권한 조정
/memory         # 프로젝트 기억 편집
```

13. Claude.ai 와 Claude Code 의 차이

지금까지 설명한 명령어들은 주로 **Claude Code(터미널)** 기준입니다. 환경별로 명령어 지원 범위가 다릅니다:

환경	슬래시 명령어
Claude.ai (웹/앱)	Plugin 명령어, 기본 명령어 일부
Claude Cowork	Plugin 명령어 (<code>/call-prep</code> , <code>/review-contract</code> 등)
Claude Code (터미널)	전체 60+ 내장 명령어 + 커스텀 명령어 전부
Claude Code (VS Code)	터미널과 동일, Ctrl+Enter로 실행

Agentic AI 기초 개념 가이드

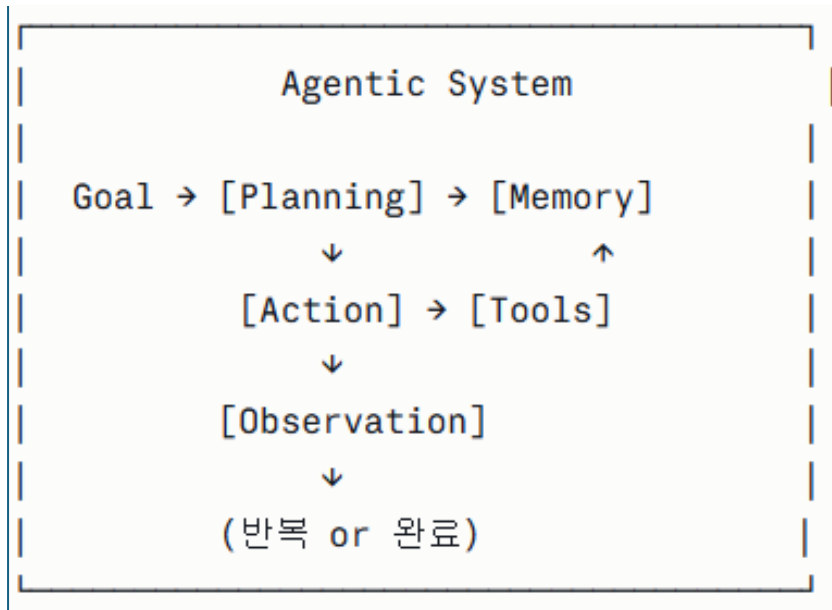
Agentic AI의 핵심 개념들을 체계적으로 설명해 드릴게요. 질문하고 싶은 개념이 많으시다면, 아래 구조를 따라가시면 됩니다.

1. Agentic AI란?

Agent(에이전트)란 단순히 질문에 답하는 것을 넘어, **목표를 받아 스스로 계획하고, 도구를 사용하고, 행동을 실행하는 AI 시스템**입니다.

구분	일반 LLM	Agentic AI
동작 방식	입력 → 출력 (1회)	목표 → 계획 → 실행 → 반복
도구 사용	없음	검색, 코드 실행, API 호출 등
자율성	낮음	높음
예시	"파이썬이란?"	"이 코드 버그 찾아서 고치고 테스트까지 해줘"

2. 핵심 구성 요소



Planning (계획)

목표를 받으면 하위 작업으로 분해합니다.

- **ReAct**: Reasoning + Acting을 번갈아 수행
- **Chain-of-Thought**: 단계별 추론
- **Tree of Thoughts**: 여러 경로를 탐색

Tools (도구)

에이전트가 실제 세계와 상호작용하는 수단입니다.

- 웹 검색, 코드 실행, 파일 읽기/쓰기
- API 호출, 데이터베이스 조회
- 다른 AI 모델 호출

Memory (메모리)

- **Short-term**: 현재 대화/작업 컨텍스트 (Context Window)
- **Long-term**: 벡터 DB, 외부 저장소
- **Episodic**: 과거 작업 기록

Observation (관찰)

도구 실행 결과를 받아 다음 행동을 결정합니다.

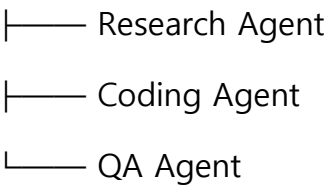
3. 주요 아키텍처 패턴

Single Agent

하나의 에이전트가 모든 작업 처리

Multi-Agent System

Orchestrator Agent



복잡한 작업을 전문 에이전트들이 협력하여 처리

Human-in-the-Loop

중요한 결정 시점에 사람이 개입하여 승인/수정

4. 대표적인 Framework

프레임워크	특징
LangGraph	그래프 기반 상태 관리, 복잡한 흐름 제어
AutoGen	멀티 에이전트 대화 중심
CrewAI	역할 기반 에이전트 팀
Claude (Anthropic)	Tool use + computer use 내장

5. 핵심 도전 과제

- **Hallucination**: 잘못된 정보로 잘못된 행동 실행
- **Long-horizon planning**: 긴 작업에서 목표 유지
- **Safety**: 돌이킬 수 없는 행동 방지

- **Cost:** 반복 LLM 호출로 인한 비용

어떤 개념부터 더 깊게 파고들어 볼까요? 예를 들어:

- **ReAct / Tool Use** 동작 원리
 - **Multi-Agent** 설계 패턴
 - **Memory** 아키텍처
 - **특정 프레임워크** (LangGraph 등)
 - **RAG와 Agent의 차이**
-

Claude Cowork 완벽 가이드

1. Cowork란 무엇인가?

Cowork는 Anthropic의 지식 업무용 에이전틱 AI 시스템입니다. 데스크톱에서 실행되며, 로컬 파일과 애플리케이션에 연결하여 다단계 작업을 처음부터 끝까지 완료합니다. 사용자가 목표를 정하면, Cowork가 거기까지 도달하는 방법을 스스로 파악합니다.

한마디로, "시키면 알아서 다 해주는 AI 동료" 입니다.

Chat vs Cowork 차이

구분	일반 Chat	Cowork
파일 접근	직접 불가 (업로드 필요)	지정 폴더 직접 읽기/쓰기
작업 방식	질문-답변 1회	목표 → 계획 → 실행 → 완료
결과물	텍스트 설명	완성된 파일 (엑셀, PPT, 문서)
지속성	짧은 대화	장시간 복잡한 작업

대부분의 AI 어시스턴트는 사용자가 작업을 개별 프롬프트로 쪼개야 하지만, Claude Cowork는 원하는 결과를 말하면 나머지를 알아서 처리합니다.

2. 탄생 배경

Anthropic 내부에서 마케팅, 데이터 등 비기술팀이 복잡한 다단계 작업(도구 빌드, 데이터 마이닝 등)을 처리하기 위해 Claude의 채팅 인터페이스를 건너뛰고 Claude Code를 직접 사용하기 시작했습니다. 같은 패턴이 외부에서도 관찰되었고, Claude Cowork는 그 결과물로 — 비개발자 지식 근로자를 위해 설계된 동일한 역량을 제공합니다.

Claude Cowork는 2026년 1월 "리서치 프리뷰"로 출시되었으며, 현재는 모든 유료 플랜에 정식 제공됩니다.

3. 핵심 기능

주요 기능은 다음과 같습니다:

- **로컬 파일 직접 접근:** 수동 업로드/다운로드 없이 로컬 파일 읽기·쓰기
- **서브 에이전트 조율:** 복잡한 작업을 소규모 작업으로 분해해 병렬 처리
- **전문가급 결과물:** 수식이 작동하는 Excel, PowerPoint 프레젠테이션, 형식 있는 문서 생성
- **장시간 작업:** 대화 타임아웃이나 컨텍스트 제한 없이 복잡한 작업 수행
- **예약 작업:** 작업을 저장하고 주기적으로 자동 실행

4. Windows 설치 방법 (단계별)

Windows 버전은 2026년 2월 10일 macOS와 동일한 기능으로 출시되었습니다.

Step 1 — 다운로드

claude.com/download에 접속하여 Windows 버전 설치 파일을 다운로드합니다. 설치 파일은 약 7MB로 매우 가볍습니다.

Step 2 — 설치

다운로드된 설치 파일을 실행하면 나머지 파일들을 자동으로 다운로드합니다. 인터넷 연결을 유지한 채 설치가 완료될 때까지 기다립니다.

⚠ **MSIX 설치 주의:** 구형 .exe/Squirrel 인스톨러 대신 MSIX 형식으로 설치해야 합니다. 만약 "VM service not running" 오류가 발생하면 공식 다운로드 페이지에서 재설치하거나, `services.msc`에서 "Claude VM Service"를 시작하면 됩니다.

Step 3 — 로그인

설치 완료 후 "시작하기" 버튼을 클릭하고, 유료 Claude 계정으로 로그인합니다.

Step 4 — Cowork 탭 진입

Claude Desktop을 열고 상단의 모드 선택기에서 "**Cowork**" 탭을 클릭합니다. 프롬프트 창에 원하는 작업을 입력하면 됩니다.

5. 실제 활용 예시

다음과 같은 실무 사용 사례들을 활용할 수 있습니다:

파일 정리

"Downloads 폴더의 모든 파일을 정리해줘.
문서, 이미지, 영상, 설치파일로 서브폴더를 만들고 이동시켜줘.
변경 전에 이동할 내용을 먼저 알려줘."

데이터 분석 + 보고서

"이 폴더의 PDF 영수증을 모두 읽어서
공급업체, 날짜, 금액, 청구서 번호를 추출하고
월별 지출 차트가 포함된 Excel 파일을 만들어줘."

복합 워크플로우

"brief.docx의 프로젝트 개요를 읽고,
관련 최신 시장 트렌드를 온라인에서 검색해서
권장사항이 담긴 분석 문서와
일정이 포함된 Excel 액션플랜을 만들어줘."

6. 요금제 및 제한

유료 플랜별로 이용 가능하며, Pro 플랜(\$20/월)은 폴더 정리나 간단한 보고서 작성 같은 빠른 작업에 적합하고, Max 플랜(\$100~200/월)은 더 길고 복잡한 작업에 적합합니다. Cowork는 Chat보다 사용량을 더 빠르게 소모합니다.

7. 보안 및 권한 구조

Cowork는 격리된 가상 머신(VM) 안에서 코드를 실행합니다. 파일 읽기/쓰기는 사용자가 연결한 폴더로만 제한되며, 네트워크 접근도 설정에 따라 제어됩니다.

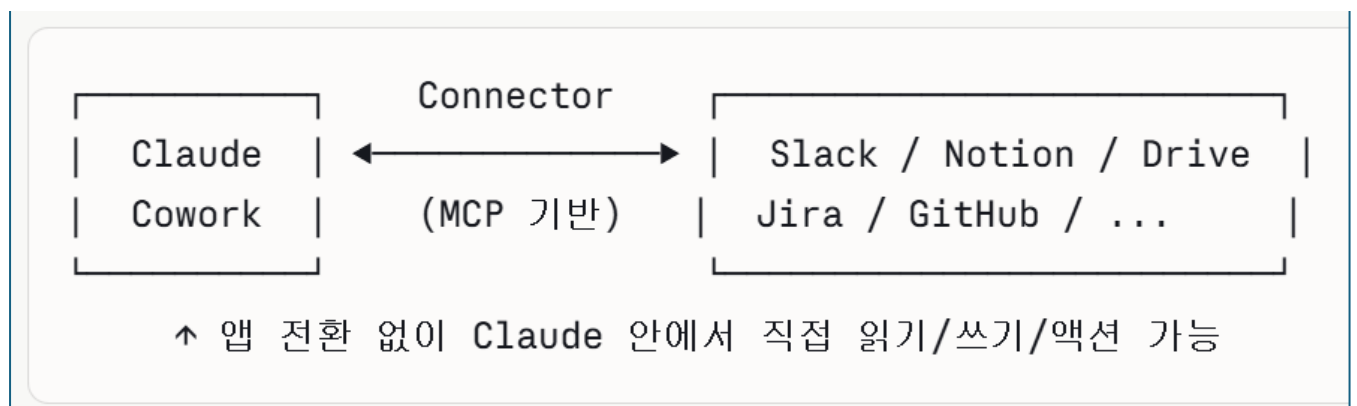
Claude가 행동하기 전에 계획을 보여주고 승인을 기다립니다. 각 단계를 실시간으로 지켜보거나, 중요한 결정 시점에 직접 개입할 수 있습니다.

Claude Connectors 완벽 가이드

1. Connector란?

Connector는 Claude가 여러분의 앱과 서비스에 접근하고, 데이터를 가져오고, 연결된 서비스 안에서 직접 행동을 취할 수 있게 해주는 기능입니다. Claude는 연결된 서비스에서 각 사람의 권한을 그대로 상속합니다. 즉, 원본 시스템에서 특정 파일, 채널, 레코드에 접근할 수 없다면 Connector를 통해서도 접근할 수 없습니다.

쉽게 말해, "**Claude와 외부 앱을 연결하는 다리**" 입니다.



2. 기술적 기반: MCP (Model Context Protocol)

Anthropic은 AI 모델이 단일 표준으로 외부 시스템에 연결할 수 있도록 MCP(Model Context Protocol)를 만들었습니다. MCP 이전에는 개발자들이 AI 어시스턴트를 연결하고 싶은 모든 앱마다 커스텀 API 커넥터를 직접 작성해야 했습니다. 이제는 MCP 서버를 한 번 만들면, 호환되는 AI 에이전트가 파일을 읽고, 함수를 실행하고, 데이터베이스를 조회할 수 있습니다.

각 Connector는 OAuth 또는 API 키를 통해 안전하게 인증하며, 기본 계정의 권한을 그대로 따릅니다. Claude는 여러분이 볼 수 있는 것만 볼 수 있습니다.

3. Connector의 두 가지 종류

🌐 Web Connector (클라우드 기반)

Web Connector는 Claude, Claude Desktop, Cowork, iOS/Android 모바일 앱 모두에서 사용 가능합니다. Google Drive, Slack, Notion처럼 클라우드 기반 서비스와 연결합니다.

 Desktop Extension (로컬 기반)

Desktop Extension은 Claude Desktop에서만 사용 가능하며, 로컬 파일 시스템, 브라우저, 네이티브 앱과 연결됩니다. 코드 서명, 민감한 데이터(API 키 등)의 암호화 저장, 기업 정책 제어 등 엔터프라이즈급 보안을 제공합니다.

4. 사용 가능한 Connector 예시

현재 Slack, Notion, Google Drive, HubSpot, Jira, Salesforce, Snowflake 등 38개 이상의 업무 도구에 연결할 수 있습니다. 모든 Connector는 무료입니다.

카테고리	서비스	가능한 작업
커뮤니케이션	Slack	메시지 검색, 채널 읽기, 메시지 전송
문서/파일	Google Drive, Notion	파일 검색, 읽기, 생성, 편집
프로젝트 관리	Jira, Linear, Asana	이슈 조회, 생성, 상태 변경
CRM/영업	Salesforce, HubSpot	고객 데이터 조회, 레코드 업데이트
데이터/DB	Snowflake, BigQuery	라이브 쿼리 실행, 대시보드 생성
크리에이티브	Blender, Adobe, Ableton	씬 편집, 디자인 자동화
생산성	Microsoft 365	Word/Excel/Outlook 연동

2026년 4월 28일, Anthropic은 Blender, Adobe, Ableton, Autodesk 등 전문 크리에이티브 소프트웨어를 위한 9개의 Connector를 추가로 출시했습니다. 무료 플랜을 포함한 모든 플랜에서 즉시 사용 가능합니다.

5. Windows에서 Connector 설정하는 방법

방법 A — 대화 중에서 추가

채팅 인터페이스 왼쪽 하단의 "+" 버튼을 클릭하거나 "/" 를 입력해 메뉴를 열고 → "Connectors" 위에 마우스를 올린 뒤 → "Manage connectors"를 선택합니다.

방법 B — 설정에서 추가

Customize > Connectors 로 이동 → "+" 버튼 클릭 → 카테고리별 또는 전체 목록에서 원하는 Connector를 선택합니다.

연결 절차 (공통)

원하는 Connector를 클릭 → 설명과 기능을 확인 → "Connect" 또는 "Install" 클릭 → 인증 프롬프트에 따라 계정 접근을 허용 → 필요한 권한과 설정을 구성합니다.

💡 한 번 인증하면 이후 세션에서도 계속 유지됩니다.

6. 실전 활용 예시

에이전트는 라이브 PostgreSQL 데이터베이스에 쿼리를 실행하거나, 활성 Jira 티켓을 읽거나, 채팅 창에서 직접 로컬 로그 파일을 확인할 수 있습니다. 실시간 컨텍스트는 AI 출력의 정확도를 높입니다.

복합 Connector 워크플로우 예시: 예를 들어, 제품 매니저는 Amplitude에서 쿼리를 가져오고, Canva 템플로 만들고, Asana에 링크를 팀을 위해 드롭하는 작업을 대화를 벗어나지 않고 한 번에 처리할 수 있습니다.

7. Custom Connector (나만의 커넥터 만들기)

Custom Connector는 원격 MCP를 사용하며 무료, Pro, Max, Team, Enterprise 모든 플랜에서 사용 가능합니다. 무료 사용자는 1개로 제한됩니다.

Custom Connector를 추가하려면 **Customize > Connectors** → "+" 클릭 → "Add custom connector" 선택 → Connector 이름과 URL 입력 → 필요시 OAuth Client ID와 시크릿 입력 후 "Add"를 클릭합니다. Custom Connector는 로컬 기기가 아닌 Anthropic의 클라우드에서 MCP 서버에 연결하므로, 서버가 공개 인터넷에서 접근 가능해야 합니다.

8. 팀/기업 환경에서의 권한 제어

Team과 Enterprise 플랜의 Owner는 연결된 서비스가 수행할 수 있는 액션을 제한할 수 있습니다. 예를 들어 이메일 검색·요약은 허용하되 메시지 전송은 차단하거나, Google Drive 파일 읽기는 허용하되 생성·편집은 차단할 수 있습니다. 각 권한 카테고리별로 "항상 허용 / 승인 필요 / 차단" 중 하나를 설정합니다.

9. Skills vs Connectors vs Plugins 차이

Skills는 개별 레시피 카드, Connectors는 주방 배관, Plugin은 완전히 갖춰진 주방 전체에 비유할 수 있습니다. Plugin은 Skills, Connectors, 슬래시 명령어, 서버 에이전트를 하나의 설치 가능한 패키지로 묶어 특정 업무 기능을 위해 설계된 것입니다.

구분	역할	예시
Skills	Claude의 능력 확장 (파일 생성 방법)	PPTX 만들기, PDF 분석
Connectors	외부 서비스와 연결 (데이터 통로)	Slack 읽기, Drive 접근
Plugins	Skills + Connectors 묶음 패키지	영업 자동화 플러그인 전체

다음으로 MCP를 직접 만드는 방법, 특정 Connector 심층 활용법, 또는 Skills 개념에 대해 더 알고 싶으신가요? 🚀

MCP (Model Context Protocol) 완벽 가이드

1. MCP란 무엇인가?

MCP(Model Context Protocol)는 AI 어시스턴트를 콘텐츠 저장소, 비즈니스 도구, 개발 환경 등 데이터가 있는 시스템에 연결하는 새로운 오픈 표준입니다. 목표는 최신 모델이 더 나은, 더 관련성 높은 응답을 생성할 수 있도록 돕는 것입니다.

가장 쉬운 비유는 이것입니다:

MCP는 LLM 생태계의 USB-C에 가장 가깝습니다. 어떤 규격의 클라이언트도 어떤 규격의 서버에든 연결할 수 있는 단일 표준으로, 앱마다 전용 케이블이 필요했던 복잡함을 해소합니다.

2. MCP가 탄생한 이유 — N×M 문제

MCP 이전의 세계를 먼저 이해해야 합니다.

MCP 이전 (N×M 문제):

```
Claude — 전용 코드 —> Slack
Claude — 전용 코드 —> Notion
Claude — 전용 코드 —> GitHub
GPT-4 — 전용 코드 —> Slack
GPT-4 — 전용 코드 —> Notion
...
```

AI 모델 수(N) × 도구 수(M) = 만들어야 할 커넥터 수

MCP 이후 (N+M 문제):

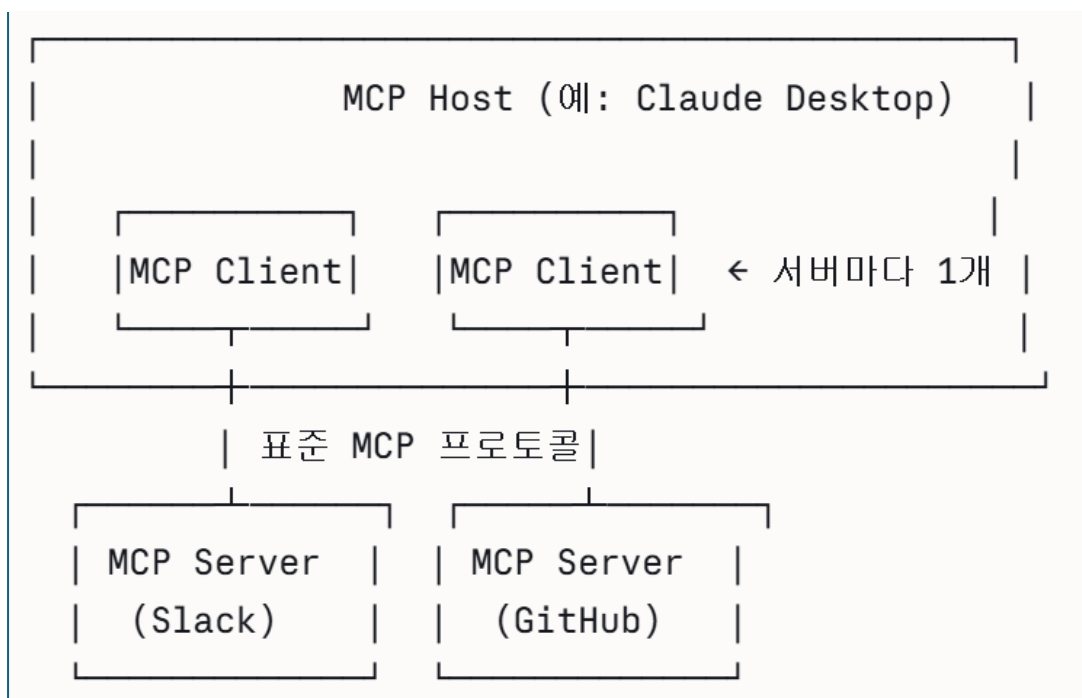
```
Claude └──────────────────┐ Slack MCP 서버
GPT-4 └─ 표준 MCP 프로토콜 ┤ Notion MCP 서버
Gemini └──────────────────┘ GitHub MCP 서버
```

AI 모델 수(N) + 도구 수(M) = 만들어야 할 것의 합

MCP 이전에 개발자들은 AI 모델을 연결하고 싶은 모든 데이터 소스나 도구마다 커스텀 커넥터를 따로 만들어야 했습니다. 이른바 "N×M 데이터 통합 문제"입니다. MCP는 JSON-RPC 2.0을 기반으로 Language Server Protocol(LSP)의 메시지 흐름 아이디어를 재활용하여 이 문제를 해결합니다.

3. MCP 아키텍처: 3개의 역할

MCP는 Host-Client-Server 구조를 통해 LLM 애플리케이션이 외부 데이터, 도구, 프롬프트와 상호작용할 수 있도록 하는 모듈형 프로토콜입니다.



Host (호스트)

MCP Host는 컨텍스트를 관리하고 실행을 제어하는 AI 애플리케이션 자체입니다(예: Claude Desktop). Host는 여러 MCP 서버에 병렬로 연결할 수 있으며, 각 서버는 자체 클라이언트로 관리됩니다.

Client (클라이언트)

Host 내부에 존재하며 MCP 서버와 1:1로 연결되어 프로토콜 메시지를 처리합니다.

Server (서버)

MCP 서버는 독립적인 프로세스로, 로컬 또는 원격에서 MCP의 표준화된 Primitive(기본 단위)를 통해 기능을 노출합니다. 서버는 집중적이고 조합

가능하며 구축하기 쉽게 설계되어 있습니다. 파일 시스템 서버는 로컬 파일을 노출하고, 데이터베이스 서버는 쿼리 기능을 제공하고, 날씨 API 서버는 외부 서비스를 래핑합니다.

4. 핵심 Primitives: 3가지 기본 단위

MCP 서버가 외부에 제공하는 능력의 종류입니다.

Tools (도구) — 모델이 실행하는 액션

Tools는 AI 애플리케이션이 액션을 수행하기 위해 호출할 수 있는 실행 가능한 함수입니다(예: 파일 작업, API 호출, 데이터베이스 쿼리).

```
python

# 예시: Slack 메시지 전송 Tool
@mcp.tool()
def send_slack_message(channel: str, message: str):
    """슬랙 채널에 메시지를 전송합니다"""
    slack_client.chat_postMessage(channel=channel, text=message)
```

Resources (리소스) — 읽기 전용 데이터

Resources는 읽기 전용 데이터 소스로, AI가 컨텍스트를 얻기 위해 읽는 것입니다(파일, 데이터베이스 레코드, 지식 베이스 등). 이들은 앱 제어 방식으로, Host가 언제 이를 노출할지 결정합니다.

예시 :

- 파일 내용: `file:///project/report.pdf`
- DB 스키마: `db://mydb/schema`
- API 응답: `api://weather/current`

Prompts (프롬프트) — 재사용 가능한 템플릿

Prompts는 재사용 가능한 상호작용 템플릿으로, AI가 서버의 도구와 리소스를 주어진 작업에 어떻게 사용할지 안내하는 사전 정의된 워크플로우 또는 지식 구조입니다. 사용자 제어 방식으로, 모델이 자율적으로 트리거하는 것이 아니라 선택 가능한 옵션으로 사용자에게 노출됩니다.

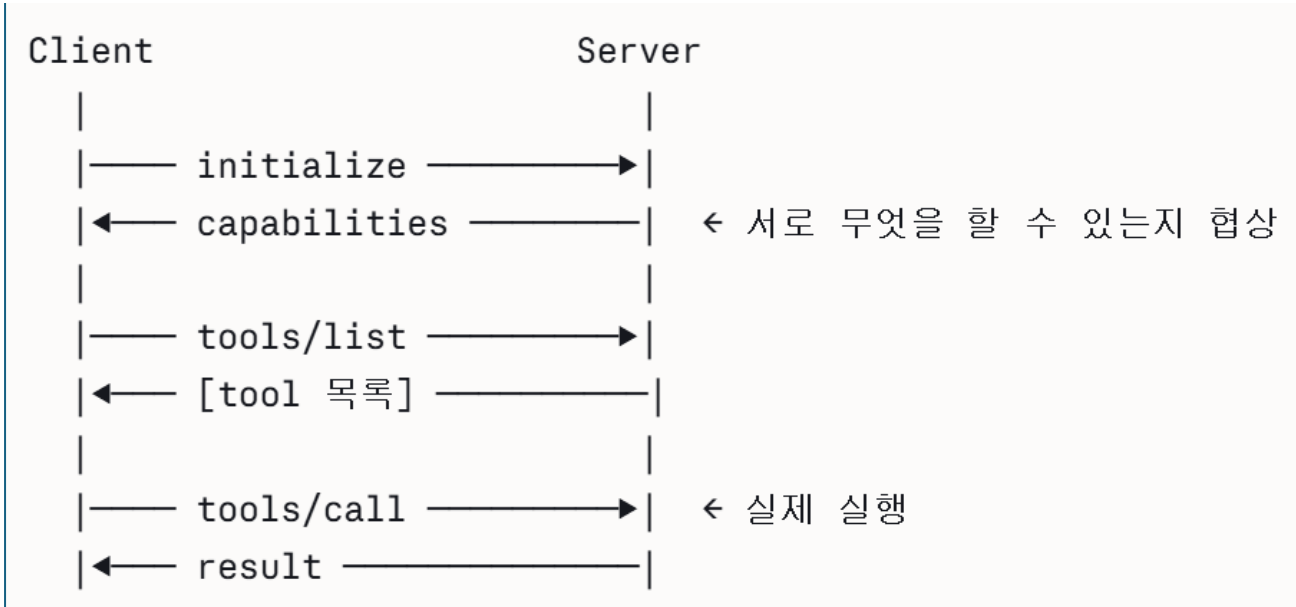
5. 통신 방식: Transport 레이어

MCP는 JSON-RPC 2.0을 사용하며 두 가지 전송 방식을 지원합니다.

방식	용도	특징
stdio	로컬 서버	표준 입출력으로 통신, 로컬 프로세스에 적합
HTTP/SSE	원격 서버	Server-Sent Events, 클라우드 서버에 적합

연결 흐름

모든 MCP 세션은 구조화된 초기화 핸드셰이크로 시작합니다. 이는 단순한 헬스 체크가 아니라 전체 세션에서 사용 가능한 기능을 결정하는 기능 협상(Capability Negotiation)입니다. 클라이언트는 지원하는 프로토콜 버전과 자체 기능을 담은 초기화 요청을 전송하고, 서버는 선택한 버전과 자신의 기능(도구, 리소스, 프롬프트, 구독 지원 여부)으로 응답합니다.



6. MCP의 확산: 업계 표준이 된 과정

Anthropic이 2024년 11월 MCP를 출시했을 때 월 SDK 다운로드 수는 약 200만 건이었습니다. OpenAI가 2025년 4월 채택하면서 2,200만 건으로 증가했고, Microsoft가 7월 Copilot Studio에 통합하면서 4,500만 건, AWS가 11월 지원을

추가하면서 6,800만 건에 달했습니다. 2026년 3월에는 공개 MCP 서버 10,000개 이상, 월 SDK 다운로드 9,700만 건을 기록했습니다.

2025년 12월, Anthropic은 MCP를 Linux Foundation 산하 Agentic AI Foundation(AAIF)에 기증했습니다. AAIF는 Anthropic, Block, OpenAI가 공동 창립했으며 Google, Microsoft, AWS, Cloudflare의 지원을 받습니다. MCP는 Kubernetes, PyTorch와 함께 Linux Foundation 포트폴리오에 이름을 올렸습니다.

7. MCP vs 일반 API vs Function Calling 차이

Function Calling은 모델 API 기능이고, MCP는 그 위의 통합 레이어입니다. 대부분의 현대 시스템은 둘 다 사용합니다.

구분	일반 API	Function Calling	MCP
표준화	없음 (각 API마다 다름)	모델별 표준	완전한 범용 표준
도구 발견	수동으로 문서 확인	스키마 수동 정의	자동 discovery
재사용성	없음	모델마다 재작성	한 번만 구현
대상	앱 개발자	AI 개발자	AI 에이전트

8. 실제 MCP 서버 만들기 (Python 예시)

```
python

from mcp.server.fastmcp import FastMCP

mcp = FastMCP("My Task Manager")

# Tool: 액션 (쓰기)
@mcp.tool()
def create_task(title: str, due_date: str) -> str:
    """새 할일을 생성합니다"""
    task_db.insert(title=title, due=due_date)
    return f"Task '{title}' created!"

# Resource: 데이터 읽기
@mcp.resource("tasks://all")
def get_all_tasks() -> str:
    """모든 할일 목록을 반환합니다"""
    return task_db.get_all()

# Prompt: 재사용 템플릿
@mcp.prompt()
def daily_review() -> str:
    """오늘 마감인 할일을 정리해서 보고해주세요"""
    return "오늘 마감인 태스크를 가져와 우선순위별로 정리해줘"

if __name__ == "__main__":
    mcp.run()
```

9. MCP 생태계 현황 (2026)

MCP는 출시 이후 월 1억 다운로드를 기록하며 AI 도구 연결의 업계 표준이 되었습니다. 현재 활용 가능한 서버들을 보면:

- **공식 서버:** Google Drive, Slack, GitHub, PostgreSQL, Puppeteer 등
- **크리에이티브:** Blender, Adobe, Ableton, Autodesk

- **비즈니스:** Salesforce, HubSpot, Jira, Linear
- **데이터:** Snowflake, BigQuery, Databricks
- **커뮤니티:** 10,000개 이상의 공개 서버

10. MCP와 Agentic AI의 관계

MCP가 Agentic AI에서 핵심인 이유를 한 문장으로 정리하면:

MCP 서버를 연결하면 Claude를 범용 채팅 인터페이스에서 특화된 팀원으로 바꿀 수 있습니다. 에이전트가 라이브 PostgreSQL 데이터베이스에 쿼리를 실행하거나, 활성 Jira 티켓을 읽거나, 채팅 창에서 직접 로컬 로그 파일을 확인할 수 있어 AI 출력의 정확도를 크게 높입니다.

다음으로 **MCP 서버를 직접 구축하는 방법**, **특정 MCP 서버 연동 실습**, 또는 ****Agentic AI의 다른 개념(RAG, Tool Use, Multi-Agent 등)****을 더 깊이 파고들어 볼까요? 🚀

Claude Skills 완벽 가이드

1. Skills란?

Skills는 Claude가 전문화된 작업에서 성능을 향상시키기 위해 동적으로 로드하는 지시사항, 스크립트, 리소스가 담긴 폴더입니다. Skills는 회사 브랜드 가이드라인으로 문서를 만들거나, 조직 특유의 워크플로우로 데이터를 분석하거나, 개인 작업을 자동화하는 등 특정 작업을 반복 가능한 방식으로 완료하는 방법을 Claude에게 가르칩니다.

가장 좋은 비유는 이것입니다:

Skills는 책장에 꽂혀 있는 요리책과 같습니다. 모든 레시피를 외울 필요가 없습니다. 필요할 때 꺼내서 따르고 다시 넣어두면 됩니다. Claude Skills도 동일하게 작동합니다. 필요할 때 로드하고, 매번 대화 컨텍스트를 차지하지 않습니다.

2. Skills의 핵심 원리: Progressive Disclosure

Skills는 점진적 공개(Progressive Disclosure) 방식으로 작동합니다. Claude가 어떤 Skill이 관련 있는지 판단하고 해당 작업을 완료하는 데 필요한 정보만 로드해서 컨텍스트 창 과부하를 방지합니다.

Claude는 세 단계로 정보를 가져옵니다:

1. **메타데이터** (이름 + 설명): 항상 Claude의 컨텍스트에 있음. 약 100 토큰. Claude는 이것만으로 Skill을 로드할지 결정합니다.
2. **SKILL.md 본문**: 트리거되었을 때만 로드됨.
3. **참조 파일들**: Claude가 본문을 읽고 필요하다고 판단할 때만 로드됨.

사용자: "이 데이터로 Excel 파일 만들어줘"



Claude: 모든 Skill의 description 읽기 (100토큰 수준)



Claude: "xlsx" Skill 관련 있다고 판단



Claude: SKILL.md 본문 로드 (나머지 Skill은 미로드)



Claude: 지시사항에 따라 Excel 생성

3. Skills의 종류

● Anthropic Skills (공식 제공)

Anthropic이 만들고 유지하는 Skills로, Excel, Word, PowerPoint, PDF 파일 생성 같은 향상된 문서 작업 Skills이 여기에 해당합니다. 모든 사용자가 사용할 수 있으며, 관련 작업 시 Claude가 자동으로 호출합니다.

현재 공식 제공되는 Anthropic Skills: docx, xlsx, pptx, pdf

● Custom Skills (사용자 직접 제작)

사용자나 조직이 특화된 워크플로우와 도메인별 작업을 위해 만드는 Skills입니다. 예를 들어 브랜드 스타일 가이드라인 적용, 회사 이메일 템플릿에 따른 커뮤니케이션 생성, 회사별 회의록 형식, Jira/Asana/Linear 작업 생성 등입니다.

● Organization Skills (조직 배포)

Team/Enterprise 플랜에서 Owner가 전체 사용자에게 Skills를 프로비저닝할 수 있습니다. 이렇게 하면 모든 팀원의 Skills 목록에 자동으로 나타나고, 기본 활성/비활성 상태를 설정할 수 있습니다.

● Partner Skills (파트너 제공)

Skills 디렉터리에는 Notion, Figma, Atlassian 등 파트너가 전문적으로 제작한 Skills이 있으며, 해당 MCP Connector와 연동되어 강력한 통합 워크플로우를 지원합니다.

4. Skill의 구조: SKILL.md 파일

모든 Skill의 핵심은 SKILL.md 파일 하나입니다.

모든 Skill은 최소한 Skill.md 파일을 포함하는 디렉터리로 구성됩니다. 이 파일은 필수 메타데이터인 name과 description 필드를 담은 YAML 프론트매터로 시작해야 합니다.

기본 구조

```
my-skill/  
├── SKILL.md           ← 핵심 파일 (필수)  
├── templates/         ← 템플릿 파일 (선택)  
│   └── report_template.md  
├── references/        ← 참조 문서 (선택)  
│   └── brand_guide.md  
└── scripts/           ← 실행 스크립트 (선택)  
    └── validate.py
```

SKILL.md 작성 형식

markdown

name: 브랜드 가이드라인

description: >

Acme Corp 브랜드 가이드라인을 발표자료나 문서에 적용합니다.
공식 색상, 폰트, 로고 사용 기준을 포함합니다.
외부 문서나 마케팅 자료 제작 시 사용하세요.

개요

이 Skill은 Acme Corp의 공식 브랜드 가이드라인을 제공합니다.

브랜드 색상

- Primary: #FF6B35 (코랄)
- Secondary: #004E89 (네이비)
- Accent: #F7B801 (골드)

타이포그래피

- 헤더: Montserrat Bold
- 본문: Open Sans Regular
- H1: 32pt / H2: 24pt / Body: 11pt

예시

****Input:**** "Q3 영업 결과 발표자료 만들어줘"

****Output:**** 위 색상과 폰트를 적용한 PowerPoint 파일

5. 트리거 메커니즘: Claude가 Skill을 어떻게 선택하나

Claude는 사용 가능한 모든 SKILL.md의 description 필드를 읽고, 이를 요청과 매칭합니다. 메시지에 Skill의 description과 일치하는 키워드나 의도가 있으면 해당 Skill이 로드됩니다. 즉, 모호한 description은 트리거 실패를 유발합니다. "문서를 도와준다"는 description은 "분기 보고서 작성"을 요청했을 때 로드되지 않을 수 있습니다.

좋은 description vs 나쁜 description:

yaml

❌ 나쁜 예 - 너무 모호함

description: "문서 관련 작업을 도와줍니다"

✅ 좋은 예 - 언제, 무엇을 하는지 명확

description: >

PDF 영수증에서 금액, 날짜, 공급업체를 추출해

Excel 지출 보고서를 생성합니다.

"영수증 처리", "지출 정리", "경비 보고서" 요청 시 사용.

6. 실전 Custom Skill 예시

예시 1: 커밋 메시지 포맷터

markdown

name: commit-message-formatter

description: >

Conventional Commits 형식으로 git 커밋 메시지를 작성합니다.
"커밋", "git 메시지", "커밋 메시지 작성" 언급 시 사용.

커밋 메시지 포맷터

모든 커밋 메시지를 Conventional Commits 1.0.0에 따라 작성합니다.

형식

<type>(<scope>): <description>

규칙

- 명령형, 소문자, 마침표 없음, 최대 72자
- Breaking change: type/scope 뒤에 `!` 추가

예시

Input: "JWT 토큰으로 사용자 인증 추가했어"

Output: `feat(auth): implement JWT-based authentication`

예시 2: 주간 보고서 생성기

markdown

name: weekly-report

description: >

팀의 주간 업무 보고서를 생성합니다.

"주간 보고서", "주보", "weekly report" 요청 시 사용.

주간 보고서 생성기

필수 섹션

1. ****이번 주 완료 사항**** - 구체적 성과 중심
2. ****진행 중인 작업**** - 현재 상태 및 예상 완료일
3. ****다음 주 계획**** - 우선순위 3가지
4. ****이슈 및 요청사항**** - 지원 필요 사항

출력 형식

- Word 문서(.docx)로 생성
- 팀장 이름을 수신인으로 표기
- 핵심 지표는 표 형식으로 정리

7. Skills 등록 및 사용 방법

Claude.ai에서 사용자 등록

Claude.ai에서 Custom Skills를 사용하려면 **Settings > Features**에서 zip 파일로 업로드합니다. Pro, Max, Team, Enterprise 플랜에서 코드 실행이 활성화된 상태에서 사용 가능합니다.

Claude Code에서 사용

Claude Code에서는 프로젝트 루트에 `skills/` 디렉토리를 만들고 그 안에 Skill 폴더를 넣으면 Claude가 자동으로 발견하고 사용합니다. API 업로드 없이 파일시스템 기반으로 동작합니다.


```
프로젝트/  
├── skills/  
│   ├── commit-message/  
│   │   └── SKILL.md  
│   └── weekly-report/  
│       └── SKILL.md  
└── src/
```

API로 등록

```
bash  
  
curl -X POST "https://api.anthropic.com/v1/skills" \  
  -H "x-api-key: $ANTHROPIC_API_KEY" \  
  -H "anthropic-beta: skills-2025-10-02" \  
  -F "display_title=My Skill" \  
  -F "files[]=@my-skill/SKILL.md"
```

8. Skills vs 다른 기능 비교

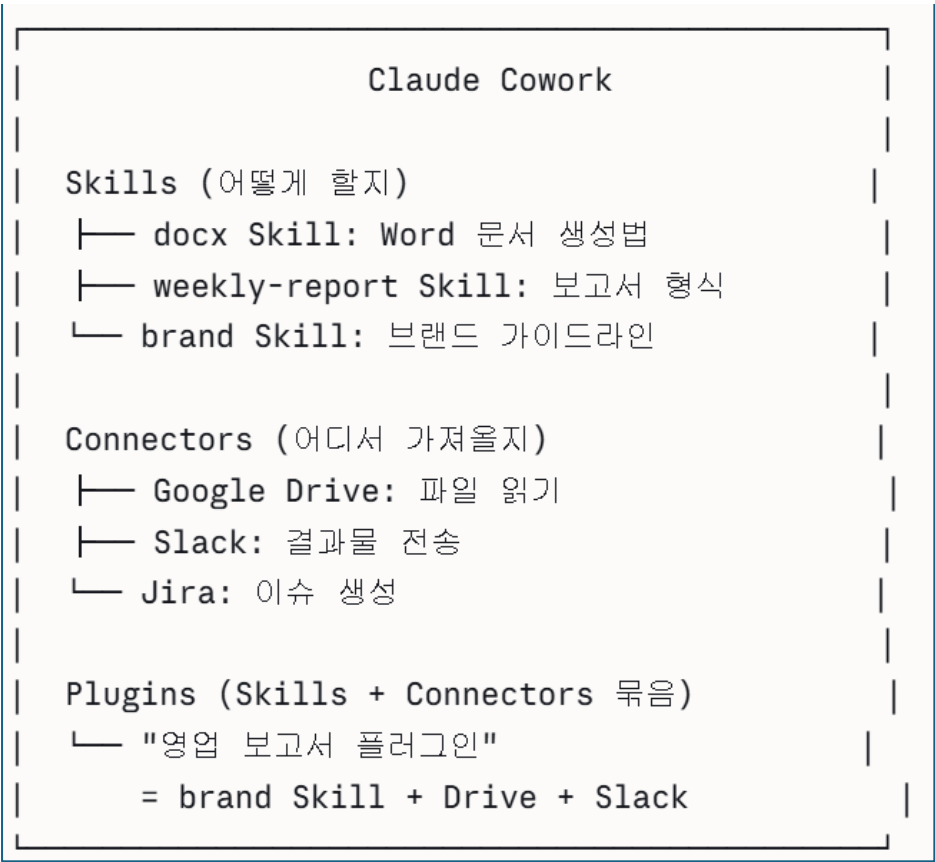
Skills와 유사 기능의 차이는 다음과 같습니다. **Projects**는 채팅 시작 시 항상 로드되는 정적 배경 지식을 제공하는 반면, Skills는 필요할 때만 동적으로 활성화되며 Claude 전체에서 작동합니다. **MCP**는 Claude를 외부 서비스와 데이터 소스에 연결하고, Skills는 절차적 지식, 즉 특정 작업을 완료하는 방법을 알려주는 지시사항을 제공합니다. 둘을 함께 사용하면 MCP가 도구 접근을 제공하고 Skills는 그 도구를 효과적으로 사용하는 방법을 가르칩니다. **Custom Instructions**는 모든 대화에 광범위하게 적용되지만, Skills는 관련될 때만 로드되어 전문화된 워크플로우에 더 적합합니다.

기능	로드 시점	범위	용도
Custom Instructions	항상	모든 대화	전반적인 선호도
Projects	프로젝트 내 항상	해당 프로젝트	배경 지식
Skills	관련 시 동적	전체 Claude	특정 워크플로우
MCP Connectors	활성화 시	전체 Claude	외부 서비스 연결

9. 오픈 표준: Agent Skills

Agent Skills 명세는 agentskills.io에서 오픈 표준으로 공개되어 있습니다. 이는 여러분이 만든 Skills가 Claude에만 종속되지 않는다는 것을 의미합니다. 동일한 Skill 형식이 이 표준을 채택하는 다른 AI 플랫폼과 도구에서도 작동합니다.

10. Skills + MCP + Connector 전체 그림



MCP는 주방 — 칼, 냄비, 재료이고, Skill은 그것들을 어떻게 사용할지 알려주는

레시피입니다. 둘을 조합할 수 있습니다. 예를 들어 Sentry의 코드 리뷰 Skill은 PR 분석 워크플로우를 Skill로 정의하고 오류 데이터는 MCP로 가져옵니다.

다음으로 ****Plugins(플러그인)****의 개념, **Sub-agents(서브 에이전트)**, 또는 **직접 Skill을 만드는 실습**으로 이어가 볼까요? 🚀

좋습니다! 공식 문서와 실제 사용 사례까지 충분히 파악했습니다. 체계적으로 설명해 드릴게요.

Claude Projects 완벽 가이드

1. Projects 란?

Projects 는 자체적인 채팅 기록과 지식 베이스를 갖춘 독립적인 워크스페이스를 만들 수 있는 기능입니다. 각 프로젝트 내에서 문서를 업로드하고, 컨텍스트를 제공하고, Claude 와 집중적인 대화를 나눌 수 있습니다.

가장 좋은 비유는 이것입니다:

Projects 는 브라우저 프로필 전환과 같습니다. 단, AI 컨텍스트를 위한 것입니다. 업무용, 사이드 프로젝트용, 개인 리서치용 등 별도의 컨텍스트 컨테이너로 분리되어, Claude 가 매 세션마다 각각에 맞는 관련 정보를 이미 알고 있는 상태로 시작합니다.

2. Projects 가 해결하는 문제

일반 Chat 의 근본적인 한계:

일반 Chat 방식 (매번 반복):

월요일: "저는 마케팅 팀장이고요, 우리 브랜드는..."

화요일: "저는 마케팅 팀장이고요, 우리 브랜드는..." ← 또 설명

수요일: "저는 마케팅 팀장이고요, 우리 브랜드는..." ← 또 설명

Projects 방식 (한 번만):

프로젝트 생성 시: 역할, 브랜드, 파일, 지시사항 한 번 설정

월요일: "Q3 캠페인 아이디어 줘" ← 바로 본론

화요일: "지난번 기획서 기반으로 수정해줘" ← 맥락 유지

수요일: "경쟁사 분석 추가해줘" ← 이전 내용 기억

모든 새 대화를 시작할 때마다 역할, 선호도, 작성 스타일, 사용 도구를 다시 설명해야 합니다. 메모리가 켜져 있어도 새 채팅은 특정 프로젝트의 맥락을 완전히 담지 못합니다. Cowork Projects 가 이 문제를 해결합니다.

3. Projects 의 3 가지 핵심 구성 요소

Claude Project

① 지식 베이스 (Knowledge Base)

업로드된 파일들 → RAG로 자동 확장

② 커스텀 지시사항 (Custom Instructions)

"항상 한국어로 답변", "법무팀 관점으로"

③ 채팅 기록 (Chat History)

프로젝트 내 모든 대화 누적 보관

① 지식 베이스 (Knowledge Base)

지식 베이스의 핵심 장점은 Claude 와의 대화에 컨텍스트를 제공하는 것입니다. 관련 문서, 텍스트, 코드 또는 기타 파일을 프로젝트의 지식 베이스에 업로드하면 Claude 가 해당 프로젝트 내 개별 채팅의 맥락과 배경을 더 잘 이해하는 데 사용합니다.

지원 파일 형식과 제한:

항목	내용
지원 형식	PDF, DOCX, CSV, TXT, HTML, 코드 파일 등
파일당 최대 크기	30MB
프로젝트 수	무료: 5개 / 유료: 무제한

② 커스텀 지시사항 (Custom Instructions)

각 프로젝트별로 프로젝트 지시사항을 정의해 Claude 의 응답을 맞춤화할 수 있습니다. 예를 들어 더 격식 있는 톤을 사용하도록 하거나 특정 역할이나 업계 관점에서 답변하도록 지시할 수 있습니다.

지시사항 예시:

당신은 법무팀 전문가입니다.

- 계약서 검토 시 항상 리스크를 먼저 언급하세요
- 법적 불확실성이 있을 때는 명확히 표시하세요
- 답변은 항상 요약 → 상세 순서로 작성하세요
- 수치와 날짜는 반드시 확인 후 제시하세요

③ 채팅 기록

긴 단일 대화는 이전 맥락을 모두 담지만, 잡담과 겹가지도 포함됩니다. Claude Projects 는 의도적으로 추가한 구조화된 컨텍스트(업로드 파일, 커스텀 지시사항)만 유지하면서 매번 깔끔한 대화를 제공합니다. 이것은 누적된 채팅 기록이 아닌 큐레이션된 지속적 메모리입니다.

4. RAG: 지식 베이스의 핵심 기술

유료 플랜 사용자는 프로젝트가 RAG(검색 증강 생성)를 통해 대용량 콘텐츠를 자동으로 처리하도록 확장됩니다. 프로젝트 지식이 컨텍스트 한계에 근접하면 Claude 가 자동으로 RAG 모드를 활성화하여 응답 품질을 유지하면서 용량을 최대 10 배까지 확장합니다.

일반 모드 (소량 문서):

파일 전체 → 컨텍스트 창에 직접 로드

[파일1 전체][파일2 전체][파일3 전체] → Claude

RAG 모드 (대용량 문서):

파일들 → 의미 기반 색인 → 질문과 관련된 부분만 검색

질문: "3분기 비용 항목은?"

→ 전체 파일 중 관련 섹션만 추출 → Claude

Claude 의 RAG 구현은 단순히 검색된 내용을 컨텍스트 토큰으로 추가하는 것이 아닙니다. 검색기가 지식 베이스에서 관련 내용을 찾고, 맥락적 세부사항을 추가하여 검색된 정보를 풍부하게 만들고, 이 강화된 컨텍스트를 사용자의 프롬프트와 결합하여 최종 응답을 생성합니다.

5. Projects 만들기 (실전 단계)

Claude.ai 웹에서 생성

1. 왼쪽 사이드바 → "Projects" → "+" 버튼
2. 프로젝트 이름 설정 (구체적으로: "2026 Q2 마케팅 캠페인")
3. 커스텀 지시사항 작성 (5~8줄 추천)
4. 지식 베이스에 관련 파일 업로드
5. 새 채팅 시작 → 바로 본론으로

Cowork 에서 생성

Cowork 탭 → Projects → "+" → 방법 선택: 처음부터 시작(Scratch), 기존 Claude Projects 에서 가져오기(Import), 또는 기존 폴더 선택 중 하나를 고릅니다.
지시사항을 5~8 줄로 간결하게 작성하세요.

6. 실전 활용 예시

예시 1: 개인 연구 프로젝트

프로젝트명: "LLM 아키텍처 연구"

지식 베이스: 논문 PDF 20개, 기술 블로그 md 파일들

지시사항: "기술적 정확성 우선. 출처 항상 명시.

불확실한 내용은 '확인 필요'로 표시."

활용: 논문 간 비교 분석, 개념 정리, 요약 보고서 생성

예시 2: 팀 영업 프로젝트 (Team 플랜)

프로젝트명: "2026 엔터프라이즈 영업 Kit"

지식 베이스: 제품 스펙, 경쟁사 분석, 고객 사례, 가격표

지시사항: "고객 관점 우선. ROI 수치 항상 포함.

법무 검토 필요 사항은 별도 표시."

공유: 영업팀 전체 → 각자 동일한 맥락으로 Claude 사용

예시 3: 소프트웨어 개발 프로젝트

프로젝트명: "MyApp 백엔드 개발"

지식 베이스: ERD, API 스펙, 코딩 컨벤션.md, README

지시사항: "Python FastAPI 기준. 타입 힌트 필수.

보안 취약점은 항상 경고 표시."

활용: 코드 리뷰, 버그 분석, 새 기능 설계

7. 협업 기능 (Team/Enterprise 플랜)

Team 과 Enterprise 플랜 사용자는 조직 구성원과 프로젝트를 공유하여 강력한 협업과 지식 공유 기능을 활성화할 수 있습니다. 권한 수준은 두 가지입니다. "사용 가능" 권한은 프로젝트 내용, 지식, 지시사항을 볼 수 있고 채팅할 수 있지만 편집은 불가능합니다. "편집 가능" 권한은 프로젝트 지시사항과 지식 수정, 구성원 추가/제거, 설정 업데이트가 가능합니다.

공유 방식:

- **개별 공유:** 특정 팀원에게 이메일로 공유
- **대량 공유:** 이메일 목록으로 여러 명 한 번에 추가
- **조직 전체 공유:** 전체 조직에 공개

8. 요금제별 차이

플랜	프로젝트 수	RAG	협업
Free	최대 5개	✗	✗
Pro (\$20/월)	무제한	✓ (10배 확장)	✗
Max (\$100~200/월)	무제한	✓	✗
Team (\$30/인/월)	무제한	✓	✓
Enterprise	무제한	✓	✓ (고급 제어)

9. Projects vs 유사 기능 완전 비교

각 기능을 언제 사용해야 하는지 정리하면 다음과 같습니다. Projects 는 **지속적인 컨텍스트**(모든 대화에 배경 지식이 필요할 때), **워크스페이스 조직**(다른 이니셔티브를 위한 분리된 컨텍스트), **팀 협업**(공유 지식과 대화 기록, Team/Enterprise), **커스텀 지시사항**(프로젝트별 톤, 관점, 접근법)이 필요할 때 선택합니다.

구분	Projects	Custom Instructions	Skills	Memory
범위	프로젝트 내	모든 대화	관련 시 동적	전체 계정
내용	문서 + 지시사항	행동 선호도	절차적 지식	개인 정보
로드	항상 (해당 프로젝트)	항상	필요 시만	항상
협업	✓ (Team+)	✗	✓ (Team+)	✗

10. 실전 팁: 효과적인 Projects 활용법

가장 흔한 실수를 피하는 방법입니다. 모든 것을 하나의 거대한 프로젝트에 넣는 것은 조직화의 목적을 무너뜨립니다. 예를 들어 마케팅 담당자라면 블로그, 소셜 미디어, 이메일 캠페인, SEO 리서치를 각각 별도 프로젝트로 유지해야 합니다. 또한 많은 사용자가 커스텀 지시사항 설정을 건너뛰고 매 채팅마다 같은 선호도를 반복해서 설명합니다. 처음에 5분을 투자해 톤, 형식, 행동 가이드라인을 정의하면 프로젝트 기간 내내 수 시간을 절약할 수 있습니다.

파일 네이밍 팁: 업로드하는 파일의 이름이 매우 중요합니다. Claude 는 먼저 파일 이름 목록을 파싱한 다음 어떤 파일이 관련 있는지 추측하고 관련 파일을 열려고 시도합니다. 파일 이름을 잘못 이해하면 많은 시간과 토큰이 낭비됩니다.

✖

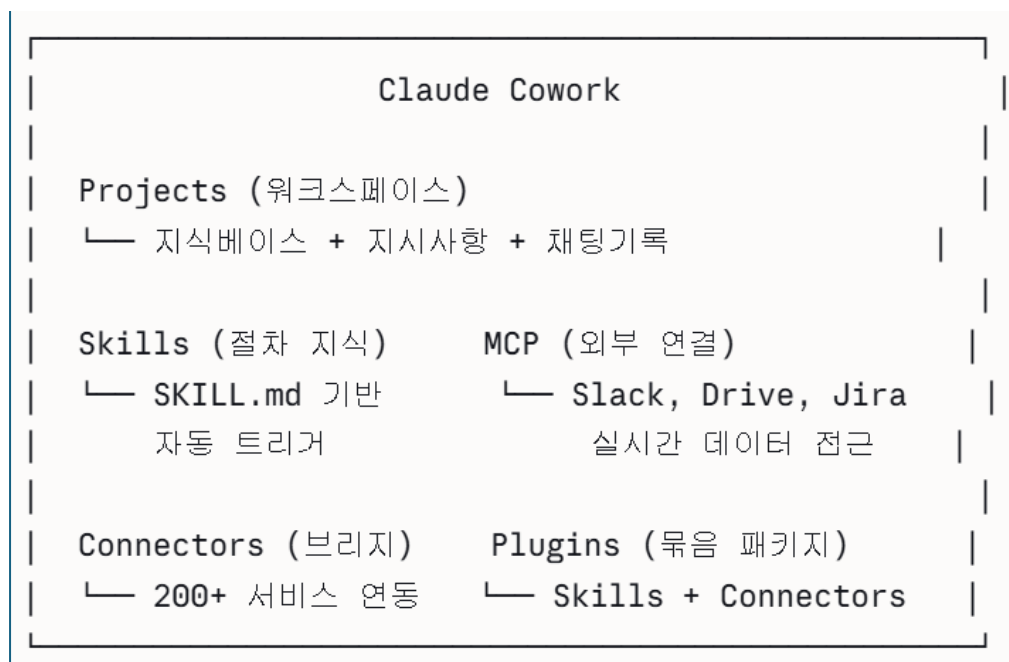
나쁜 파일명: 최종본(3).pdf, 문서1.docx, temp.csv

✔

좋은 파일명: 2026_Q2_마케팅_전략.pdf,
브랜드_가이드라인_v2.docx,
고객_인터뷰_데이터_2026.csv

전체 Claude 생태계 한눈에 보기

지금까지 배운 개념들을 하나로 정리하면:



다음으로 **Plugins(플러그인)**, **Sub-agents(서브 에이전트)**, **Memory 시스템**, 또는 이 모든 개념을 통합한 **실전 워크플로우 설계**를 이어서 알아볼까요? 🚀

Claude Cowork Plugins 완벽 가이드

1. Plugins란?

Plugins는 Skills, Connectors, Sub-agents를 하나의 패키지로 묶어서 각 역할, 팀, 회사에 맞게 Claude가 작동하는 방식을 커스터마이징합니다. 각 구성 요소를 개별적으로 설정하는 대신 Plugin을 설치하는 것만으로 첫 대화부터 바로 사용 가능한 설정을 얻을 수 있습니다.

앞서 배운 개념들과 비교하면 이렇습니다:

Skills는 지시사항이고, Connectors는 배관이라면, Plugins은 완전히 갖춰진 주방 전체입니다. Skills, Connectors, 명령어를 하나의 설치 가능한 툴킷으로 묶어 특정 직무 기능을 위해 설계된 것입니다.

2. Plugin이 해결하는 문제

Plugin 없이 (매주 반복):

1. "허브스팟에서 이번 주 리드 가져와줘"
 2. (결과 복사해서 붙여넣기)
 3. "이 형식으로 콜 브리핑 만들어줘"
 4. (내 브랜드 가이드라인 설명)
 5. "경쟁사 최신 뉴스도 포함해줘"
 6. (Gmail에 직접 복사해서 초안 작성)
- 30~40분 소요

Sales Plugin 설치 후:

/call-prep Acme Corp

- HubSpot에서 자동 조회 + 경쟁사 분석 +
내 브랜드 톤으로 브리핑 작성 + Gmail 초안 저장
- 3분 완료

Plugin을 설치하면 워크플로우를 한 번만 설정하면 됩니다. 그 시점부터 하나의 명령어가 전체 에이전틱 프로세스를 트리거하고 완성된 결과물을 돌려줍니다.

3. Plugin의 4가지 구성 요소

Plugin은 하나의 매니페스트 파일(plugin.json)로 모든 것이 연결됩니다.

```

my-plugin/
├── .claude-plugin/
│   └── plugin.json          ← ① 매니페스트 (플러그인 정의)
├── .mcp.json                ← ② MCP 도구 연결 (Connectors)
├── commands/               ← ③ 슬래시 명령어 (명시적 실행)
│   ├── call-prep.md
│   └── weekly-report.md
└── skills/                 ← ④ 도메인 지식 (자동 트리거)
    ├── crm-workflow/
    │   └── SKILL.md
    └── email-style/
        └── SKILL.md

```

① plugin.json (매니페스트)

```

json
{
  "name": "sales-plugin",
  "version": "1.0.0",
  "description": "영업팀 전용 워크플로우 자동화",
  "skills": ["skills/crm-workflow", "skills/email-style"],
  "commands": ["commands/call-prep", "commands/weekly-report"],
  "connectors": ["hubspot", "gmail", "slack"]
}

```

② Connectors (외부 서비스 연결)

Plugin 안에 필요한 Connector들이 미리 설정되어 있어 개별 설정 불필요

③ 슬래시 명령어 (명시적 실행)

각 마크다운 파일이 하나의 명령어를 정의합니다. 예: `/plugin:send-updates`.
사용자가 직접 타이핑해서 실행합니다.

④ Skills (자동 트리거)

Skills는 Claude가 무언가를 실행하기 전에 읽는 지시사항으로, 최선의 방법을

알고 있는 상태에서 작업을 수행합니다.

4. Anthropic 공식 Plugin 11종 전체 분석

2026년 1월 30일, Anthropic은 Claude Cowork에 Plugin 기능을 추가하고 팀이 사용하는 11개의 플러그인을 오픈소스로 공개했습니다. 이 플러그인들은 Skills, 슬래시 명령어, MCP Connector, Sub-agents가 특정 도메인에서 탁월하도록 구성된 완전한 패키지입니다.

● Productivity (생산성)

목적: 작업, 캘린더, 일일 워크플로우 관리

주요 명령어:

`/update` → 이메일/캘린더/채팅을 스캔해

내일 우선순위 작업 목록 자동 업데이트

`/competitive-brief` → 경쟁사 브리핑 자동 생성

Connectors: Google Calendar, Slack, Gmail

활용: "오늘 이메일과 슬랙 메시지를 스캔해서

내일 우선순위 목록 업데이트해줘"

● Sales (영업)

목적: CRM 연동 + 전체 영업 사이클 관리

주요 명령어:

`/call-prep` → 미팅 전 브리핑 자동 생성

`/follow-up` → 미팅 후 팔로업 이메일 작성

Connectors: HubSpot, Salesforce, Gmail, LinkedIn

실전 예시:

"내일 Acme Corp 콜 준비해줘:

최근 CRM 인터랙션 5개, 최근 뉴스,

경쟁사 대비 장점, 클로징 제안 포함"

● Legal (법무)

목적: 계약서 검토와 컴플라이언스 가속화

주요 명령어:

`/review-contract` → GREEN/YELLOW/RED 플래그로

조항별 계약서 분석

`/triage-nda` → NDA 신속 사전 심사

(표준 승인/법무 검토/전체 검토)

활용: 계약서 PDF 업로드 후 `/review-contract` 실행

→ 안전(초록), 위험(노랑), 치명적(빨강) 자동 분류

● Finance (재무)

목적: 스프레드시트 분석 + 재무 보고서 준비

주요 기능:

- Excel 데이터 분석 후 PowerPoint로 자동 변환
- 재무 모델 검토 및 요약

Connectors: Excel, PowerPoint, FactSet, MSCI

● Marketing (마케팅)

목적: 콘텐츠 캘린더 + 캠페인 성과 관리

주요 기능:

- 캠페인 보고서 자동 생성
- 브랜드 보이스 일관성 유지
- 다채널 콘텐츠 초안 작성

Connectors: Google Analytics, HubSpot, Canva

● Customer Support (고객 지원)

목적: 티켓 트리아지, 응답 초안, 에스컬레이션

Connectors: Slack, Intercom, HubSpot, Guru,
Jira, Notion, Microsoft 365

● Product (프로덕트)

목적: 스펙 작성, 로드맵, 사용자 리서치 합성

Connectors: Slack, Linear, Asana, Figma,
Amplitude, Pendo, Jira, Notion

○ Engineering (엔지니어링)

목적: 문서화, 디버깅, 개발 워크플로우

주요 기능:

- 코드 리뷰 자동화
- 기술 문서 생성
- PR 요약 및 분석

● Data Analysis (데이터 분석)

목적: DB 쿼리 + 대시보드 생성

주요 명령어:

`/write-query` → 자연어로 SQL 쿼리 생성

`/dashboard` → 분석 결과 시각화

Connectors: Snowflake, BigQuery, Databricks

◆ Enterprise Search (엔터프라이즈 검색)

목적: 조직 전체 문서 통합 검색

주요 기능:

- 여러 소스를 동시에 검색
- 관련 문서 자동 서피싱

Connectors: Google Drive + Slack + Notion +
SharePoint + GitHub 동시 연결

🔧 Plugin Create (플러그인 제작기)

목적: 커스텀 플러그인을 Claude와 함께 제작

주요 기능:

- 자연어로 플러그인 설계 가이드
- 템플릿 기반 빠른 시작
- Skill/Command/Connector 자동 구성

5. Plugin 설치 및 사용 방법

설치 단계

1. Claude Desktop 앱을 열고 "Cowork" 탭으로 전환합니다.
 2. 왼쪽 사이드바의 "Customize" 메뉴를 클릭합니다.
 3. "Browse plugins"를 클릭해 사용 가능한 옵션을 확인합니다.
 4. 원하는 Plugin의 "Install"을 클릭합니다.
- 직접 만든 커스텀 플러그인 파일을 업로드할 수도 있습니다.

사용 방법

각 Plugin을 설치하면 Cowork에서 사용할 수 있는 Skills가 추가됩니다. "/"를 입력하거나 "+" 버튼을 클릭하면 설치된 Plugin의 사용 가능한 Skills를 확인할 수 있습니다.

자동 트리거 (Skills):

"이 계약서 검토해줘" → Legal Plugin의 review-contract Skill 자동 실행

명시적 실행 (슬래시 명령어):

/call-prep Acme Corp → 영업 콜 브리핑 즉시 생성

/write-query 월별 매출 추이 → SQL 자동 생성

6. Plugin 커스터마이징

Plugin을 설치한 후 워크플로우에 맞게 맞춤 설정할 수 있습니다. 설치된 Plugin을 보는 중에 오른쪽 상단의 "Customize"를 클릭하면 Claude와 협업해 해당 Plugin의 Skills와 Connectors를 작업 방식에 맞게 조정하는 새로운 Cowork 작업이 열립니다.

예를 들어 Sales Plugin을 회사에 맞게 커스터마이징:

"Sales Plugin을 우리 회사 방식으로 수정해줘:

- CRM은 HubSpot 대신 Salesforce 사용
- 콜 브리핑은 한국어로 작성
- 경쟁사는 항상 A사, B사, C사와 비교
- 이메일 톤은 격식체 유지"

7. 커스텀 Plugin 직접 만들기

방법 1: Plugin Create 사용 (권장, 코딩 불필요)

Cowork에서:

"영업 후 고객 팔로업 자동화 플러그인을 만들어줘.

Notion에서 미팅 노트를 읽고,

우리 이메일 톤으로 팔로업 초안을 작성하고,

Gmail 임시 보관함에 저장하는 기능이 필요해."

→ Claude가 `plugin.json` + `SKILL.md` + `commands/*.md`
전체 구조를 자동으로 생성

방법 2: 파일 직접 작성

```
markdown
```

```
<!-- commands/client-updates.md -->
```

```
---
```

```
name: client-updates
```

```
description: 모든 활성 클라이언트에게 주간 업데이트 전송
```

```
---
```

```
# 클라이언트 주간 업데이트
```

```
## 단계
```

1. Notion에서 이번 주 완료된 작업 조회
2. 각 클라이언트별로 진행 상황 요약
3. 우리 커뮤니케이션 스킬 가이드라인 적용
4. Gmail 임시 보관함에 초안 저장
5. 검토를 위한 초안 목록 출력

방법 3: GitHub에서 포크

Anthropic은 플러그인 컬렉션을 GitHub에 오픈소스로 공개했습니다. 무엇이든 포크하고 수정할 수 있습니다.

```
bash
```

```
git clone https://github.com/anthropics/knowledge-work-plugins
```

```
cd knowledge-work-plugins/legal/
```

```
# 파일 수정 후 zip으로 압축해 Cowork에 업로드
```

8. 조직(Enterprise) Plugin 관리

Team 또는 Enterprise 플랜이라면 Owner가 플러그인 마켓플레이스를 통해 조직 전체에 Plugin을 배포할 수 있습니다. 조직 관리 Plugin은 편집이 불가능하며,

이는 팀 전체의 공유 도구를 일관되게 유지합니다. 일부 Plugin은 자동 설치되거나 필수로 설정될 수 있습니다.

Enterprise 조직 Plugin 배포 흐름:

Admin → 플러그인 마켓플레이스 구성

↓

법무팀: Legal Plugin (필수 설치)

영업팀: Sales Plugin (자동 설치)

마케팅: Marketing Plugin (선택 설치)

엔지니어: Engineering Plugin (선택 설치)

각 팀원 → 승인된 Plugin 목록에서 선택/설치

(그룹별로 다른 Plugin 카탈로그 표시 가능)

9. Plugin이 만든 시장 파급 효과

다음 날 소프트웨어 주식에서 2,850억 달러의 시가총액이 증발했습니다. Thomson Reuters는 사상 최악의 하루를 기록했고, LegalZoom, Wolters Kluwer, ServiceNow, Adobe 모두 타격을 받았습니다. Jefferies 애널리스트들은 이를 "SaaSpocalypse"라고 불렀습니다. 이 모든 것이 새로운 AI 모델이나 추론의 돌파구 때문이 아니었습니다. Plugin 때문이었습니다.

Legal Plugin 하나가 계약 검토 소프트웨어 시장 전체를 뒤흔든 이유:

기존 방식:

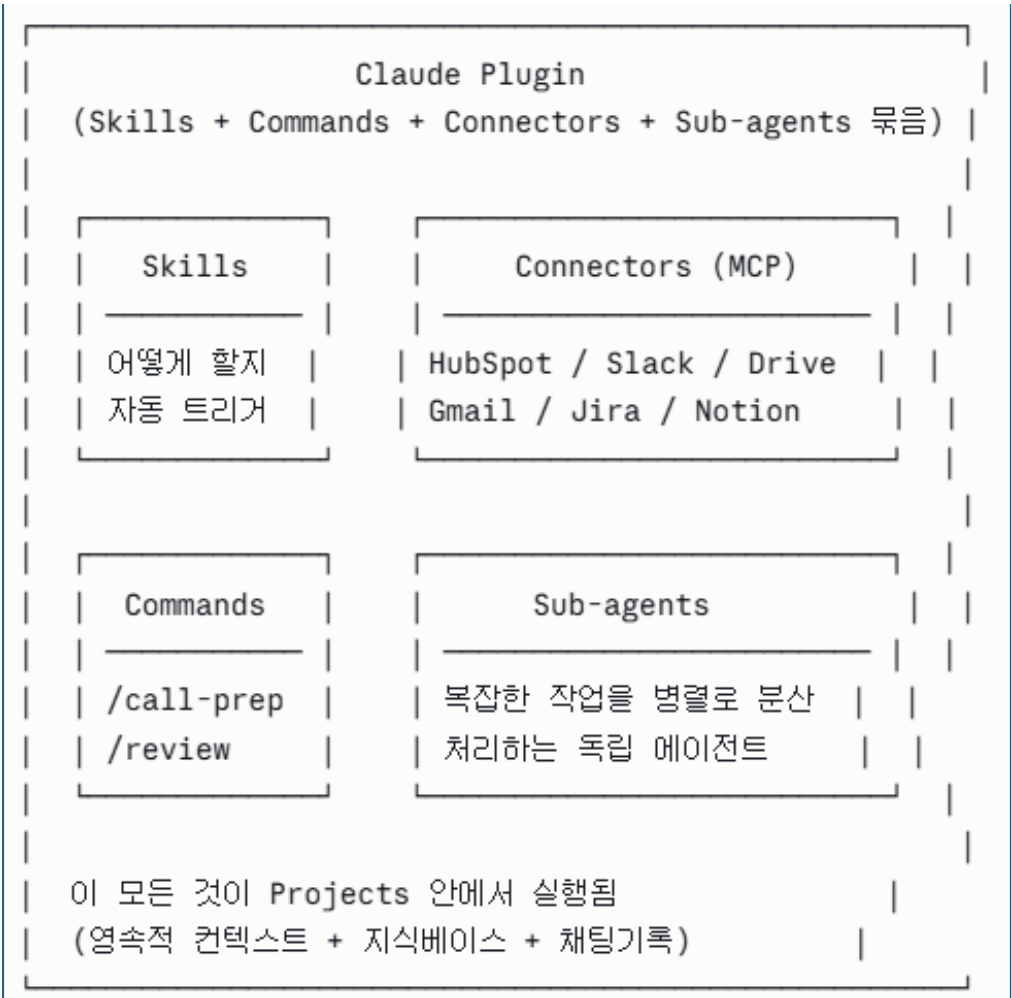
계약서 검토 소프트웨어 (월 수백만 원) + 전문 인력

Plugin 방식:

/review-contract → GREEN/YELLOW/RED 자동 분류
(Claude Pro \$20/월 포함)

10. 전체 생태계 최종 정리

지금까지 배운 모든 개념을 하나의 그림으로:



기능	역할	비유
MCP	표준 연결 프로토콜	USB 규격
Connectors	외부 서비스 연결	USB 케이블
Skills	작업 수행 지식	레시피
Projects	영속 워크스페이스	사무실
Plugins	모든 것의 묶음	완비된 팀

다음으로 **Sub-agents**(서브 에이전트), **Scheduled Tasks**(예약 작업), 또는 이 모든 개념을 통합한 **실전 엔터프라이즈 워크플로우 설계**로 이어갈까요? 🚀

Agent & Subagent 완벽 가이드

1. Agent란?

지금까지 배운 Claude의 모든 기능(Tools, MCP, Skills, Connectors, Projects, Plugins)은 결국 **Agent를 구성하는 부품들이었습니다**. Agent는 이 모든 것을 통합해 실제로 동작하는 주체입니다.

Agent는 단순히 질문에 답하는 것이 아니라 목표를 받아 자율적으로 계획하고 다단계 작업을 실행합니다. 오케스트레이터 Agent는 여러분의 목표를 받아 계획을 수립하고 실행을 관리하는 기본 인스턴스입니다.

일반 Chat:

사용자 질문 → Claude 응답 (1회 왕복)

Agent:

사용자 목표 → 계획 → 도구 실행 → 관찰 → 재계획 → ... → 완성된 결과물
↑ 피드백 루프 ↑

2. Subagent란?

Subagent는 특정 유형의 작업을 처리하는 전문화된 AI 어시스턴트입니다. 부수적인 작업이 검색 결과, 로그, 파일 내용 등으로 메인 대화를 넘쳐나게 만들 때 사용합니다. Subagent가 해당 작업을 자체 컨텍스트 내에서 수행하고 요약만 반환합니다.

Subagent는 이름이 있는 독립된 Claude 인스턴스로, 자체 시스템 프롬프트, 자체 컨텍스트 창, 자체 도구 접근 목록, 자체 권한 모드를 가집니다. 메인 Agent가 작업을 Subagent에 위임하면, Subagent는 독립적으로 실행되고 최종 출력만 부모에게 반환합니다. 파일 읽기, 검색 결과, 탐색적 도구 호출 등 모든 중간 노이즈는 Subagent의 컨텍스트 안에 머물며 메인 대화에는 절대 닿지 않습니다.

3. 왜 Subagent가 필요한가? — 컨텍스트 창 문제

모든 AI 모델에는 최대 컨텍스트 창이 있습니다. 복잡한 작업은 종종 단일 컨텍스트 창에 담기는 것보다 더 많은 정보를 필요로 합니다. 여러 에이전트에

작업을 분산함으로써 각 에이전트가 자체 컨텍스트 창 내에서 운영되어 팀 전체가 훨씬 더 큰 작업을 집합적으로 처리할 수 있습니다.

단일 Agent로 대규모 코드베이스 분석 시도:

[파일 100개 읽기][테스트 실행 로그][검색 결과][분석 중...]
→ 컨텍스트 창 한계 도달 → 품질 저하, 실패

Subagent 분산 처리:

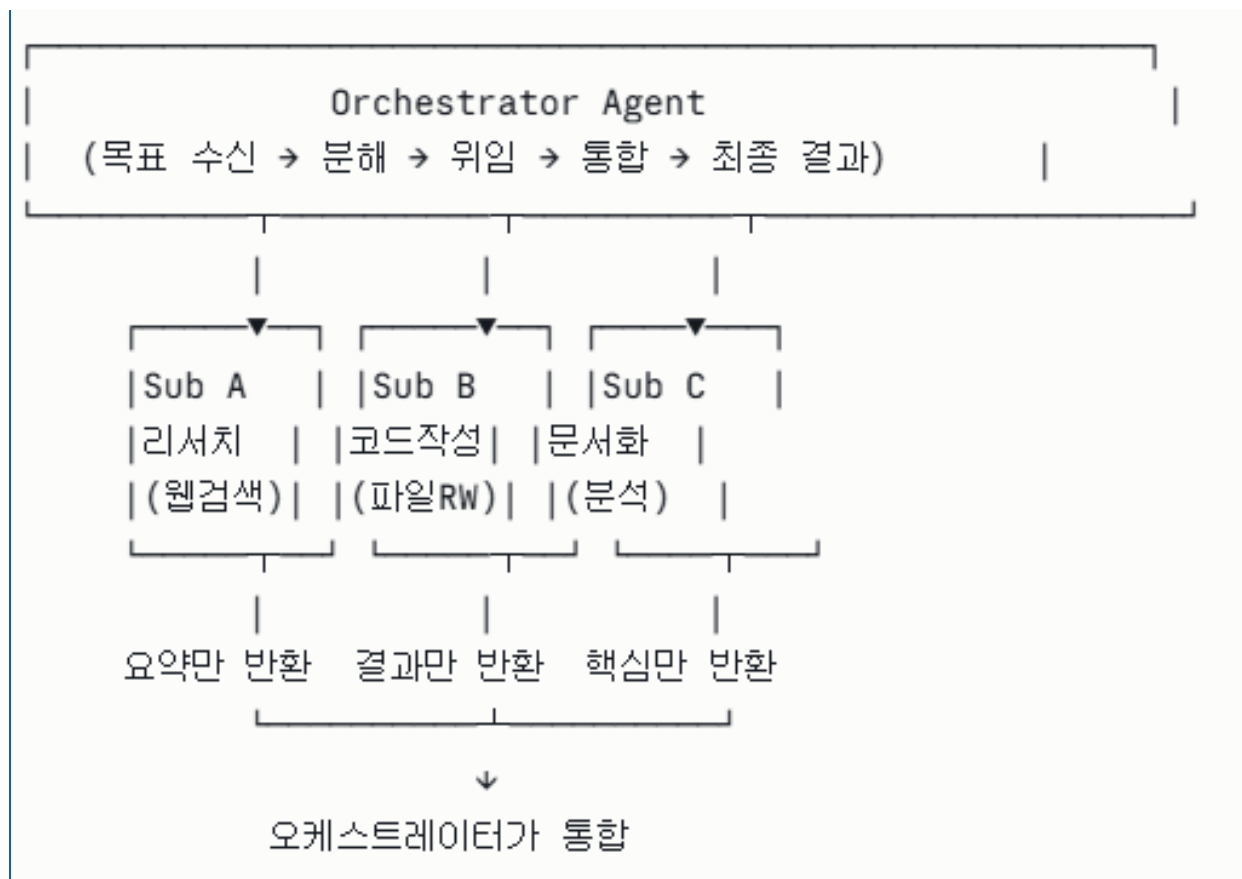
메인 Agent (계획 + 통합)

- └─ Subagent A: 파일 탐색 → 요약만 반환
- └─ Subagent B: 테스트 실행 → 결과만 반환
- └─ Subagent C: 문서 분석 → 핵심만 반환

→ 각 Subagent: 깨끗한 컨텍스트로 전문 작업 수행
→ 메인 Agent: 요약들만 받아 통합

4. Agent 아키텍처: Orchestrator와 Subagent 관계

Claude Agent Teams는 주 Agent(오케스트레이터)가 복잡한 작업을 분해하고 전문화된 Subagent에 부분 작업을 위임하는 멀티 에이전트 아키텍처입니다. 마치 컨설팅 팀과 같습니다. 프로젝트 매니저(오케스트레이터)가 클라이언트 브리프를 받아 한 분석가에게 리서치를, 다른 분석가에게 재무 모델링을, 또 다른 사람에게 작성을 맡기는 것처럼 작동합니다.



5. Subagent의 3가지 핵심 특성

① 독립적 컨텍스트 창

각 Subagent는 자체 컨텍스트 창을 가지며, 부모가 대용량 검색 결과나 장황한 출력으로 압도되는 것을 막습니다.

② 병렬 실행

일부 부분 작업은 동시에 실행될 수 있습니다. 리서치 에이전트가 정보를 수집하는 동안 계획 에이전트가 구조를 잡고, 그 후 작성 에이전트가 둘 다 결합합니다. 이 병렬성이 총 작업 완료 시간을 극적으로 단축합니다.

③ 전문화

다른 Subagent에게 다른 시스템 프롬프트, 도구, 전문화를 부여할 수 있습니다. 리서치 에이전트는 철저하고 회의적으로 설정되고, 작성 에이전트는 특정 브랜드 보이스에 맞게, 코드 에이전트는 실행 도구에 접근하도록 설정될 수 있습니다.

6. 내장 Subagent 종류 (Claude Code 기준)

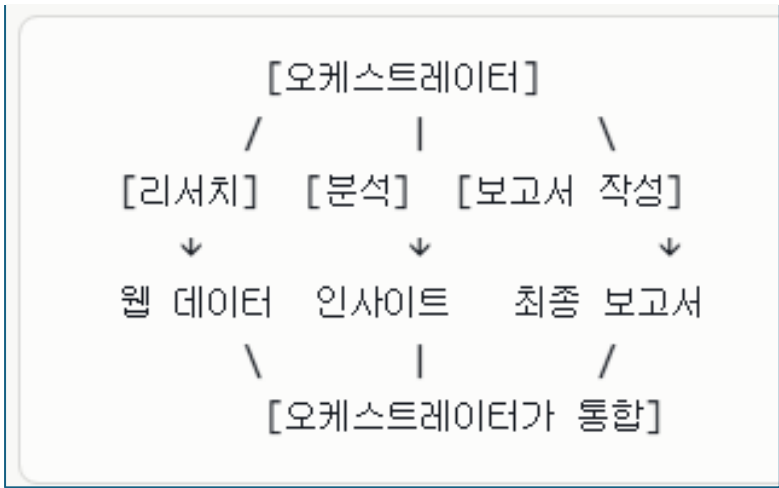
Claude Code는 여러 전문화된 내장 Subagent 유형을 제공합니다:

Subagent	역할	특징
Explore	코드베이스 탐색	파일 검색, 아키텍처 파악
Plan	아키텍처 설계	구현 계획, 트레이드오프 분석
Bash	명령어 실행	git, npm, docker, 테스트 실행
General-purpose	복합 작업	탐색+수정이 동시에 필요한 작업

7. 실전 Multi-Agent 패턴

패턴 1: Hub-and-Spoke (허브 & 스포크)

가장 일반적인 구조. 오케스트레이터가 중앙에서 모든 Subagent를 제어합니다.



실전 예: 시장 조사 보고서

목표: "전기차 시장 종합 분석 보고서 작성"

오케스트레이터:

└─ Subagent 1 (리서치): 최신 시장 데이터 웹 검색

└─ Subagent 2 (경쟁사): 주요 플레이어 분석

└─ Subagent 3 (재무): 매출/성장률 데이터 수집

└─ Subagent 4 (작성): 수집된 데이터로 보고서 초안

오케스트레이터: 4개 결과 통합 → 최종 편집 → 완성

패턴 2: Pipeline (파이프라인, 순차 처리)

한 Subagent의 출력이 다음 Subagent의 입력이 됩니다.

[데이터 수집] → [정제/분석] → [보고서 생성] → [리뷰]



raw 데이터



정제된 데이터



초안 보고서



최종본

패턴 3: Split-and-Merge (분산 후 통합)

독립적인 병렬 처리 후 통합합니다.

[오케스트레이터]

/

\

[팀A 코드 리뷰]

[팀B 테스트 생성]

← 동시 실행

\

/

[결과 통합 + 충돌 해결]

패턴 4: Specialist Team (전문가 팀)

대규모 코드 리팩토링: 코드베이스를 모듈로 나누고 각 모듈을 Subagent에 배정합니다. 하나는 테스트를 재작성하고, 다른 하나는 타입 정의를 업데이트하고, 또 다른 하나는 API 레이어를 처리합니다. 오케스트레이터가 변경 사항을 통합하고 충돌을 확인합니다.

[보안 전문 Subagent] → 취약점 스캔
[성능 전문 Subagent] → 병목 지점 파악
[UX 전문 Subagent] → 접근성 검토
[문서 전문 Subagent] → API 문서 생성
↓
[오케스트레이터]: 종합 품질 보고서

8. 커스텀 Subagent 만들기

동일한 종류의 작업자를 같은 지시사항으로 계속 생성하게 된다면 커스텀 Subagent를 정의하세요. 각 Subagent는 자체 컨텍스트 창에서 커스텀 시스템 프롬프트, 특정 도구 접근, 독립적 권한과 함께 실행됩니다.

```
yaml

# subagents/security-reviewer.yml
name: security-reviewer
description: >
  보안 취약점 분석 전문가. 코드 변경이나
  새 파일이 있을 때 OWASP 기준으로 검토.
system_prompt: |
  당신은 보안 전문가입니다.
  OWASP Top 10 기준으로 코드를 검토합니다.
  발견사항은 심각도(HIGH/MED/LOW)와 함께 보고합니다.
allowed_tools:
  - Read
  - Grep
  - WebSearch
permission_mode: read-only # 파일 수정 불가
model: claude-haiku-4-5 # 빠른 모델로 비용 절감
```

```
yaml

# subagents/doc-writer.yml
name: doc-writer
description: >
  기술 문서 작성 전문가. 코드 변경 후
  README, API 문서, 주석 자동 업데이트.
system_prompt: |
  당신은 기술 문서 작가입니다.
  코드를 분석해 명확하고 실용적인 문서를 작성합니다.
  항상 예제 코드를 포함합니다.
allowed_tools:
  - Read
  - Write
  - Edit
model: claude-sonnet-4-6
```

9. Subagent vs Agent Teams 차이

Subagent는 단일 세션 내에서 작동합니다. Agent Teams는 별도의 세션 간에 조율합니다.

구분	Subagent	Agent Teams
통신 방식	부모→자식 일방향	서로 직접 소통 가능
세션	단일 세션 내	독립 세션 간 협업
안정성	안정적, 검증됨	실험적 기능
사용 시기	작업이 독립적일 때	에이전트가 서로 발견 공유 필요할 때
복잡도	낮음	높음

대부분의 개발 작업에서 Subagent가 올바른 답입니다. 안정적이고 잘 이해되며,

격리 모델이 엔지니어가 이미 생각하는 작업 분해 방식과 깔끔하게 매핑됩니다.

10. 안전: Human-in-the-Loop

Agent와 Subagent에서 가장 중요한 원칙입니다.

고위험 행동, 즉 클라이언트에게 이메일 보내기, 레코드 삭제, 결제 진행 같은 경우에는 Human-in-the-Loop AI 오케스트레이션이 에이전트가 멈추고 진행 전에 허가를 요청합니다.

자동 실행 가능 :	승인이 필요한 것 :
 파일 읽기	 파일 영구 삭제
 웹 검색	 이메일/슬랙 전송
 코드 분석	 데이터베이스 수정
 보고서 초안 작성	 외부 API 결제 호출
 로컬 파일 정리	 대규모 파일 변경

Anthropic의 2026년 보고서에 따르면, 개발자들은 업무의 약 60%에서 AI를 사용하지만 "완전 위임"이 가능하다고 보고하는 것은 0~20%에 불과합니다. AI는 상시 협업자로 기능하지만 효과적인 사용에는 여전히 신중한 설정, 프롬프팅, 적극적인 감독, 검증, 인간의 판단이 필요합니다.

11. Cowork에서 Subagent 실전 활용

Claude가 복잡한 작업을 받으면 동시에 여러 부분을 병렬로 처리하는 Sub-agents(병렬 작업자)를 생성할 수 있습니다.

실전 Cowork 예시:

사용자: "2025년 연간 보고서 준비해줘.

재무 데이터는 Excel에, 경쟁사 분석은

웹에서 찾고, 최종본은 PowerPoint로."

Cowork 오케스트레이터 (자동 분해):

└─ Subagent 1: Excel 파일 읽어서 재무 분석

└─ Subagent 2: 웹 검색으로 경쟁사 최신 정보 수집

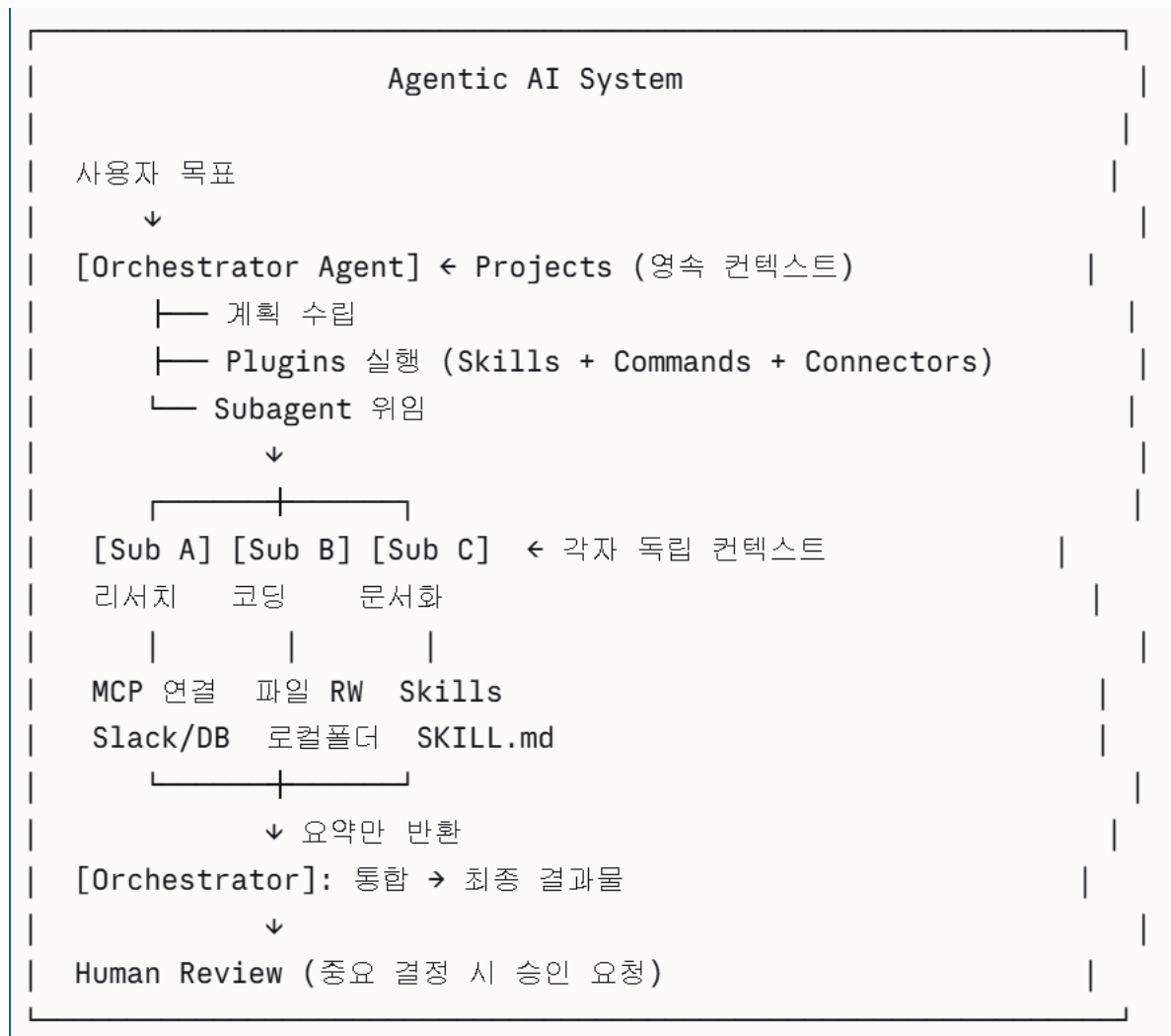
└─ Subagent 3: 회사 브랜드 가이드라인 파일 분석

오케스트레이터: 3개 결과 통합

→ PowerPoint Skills로 최종 슬라이드 생성

→ 검토 요청

12. 지금까지 배운 모든 개념의 최종 통합



다음 주제로는 **Memory 시스템** (단기/장기 메모리), **Computer Use** (화면 제어), **Scheduled Tasks** (예약 작업), 또는 **실전 멀티 에이전트 시스템 구축 실습**으로 이어가볼까요? 🚀

Tool Use & Computer Use 완벽 가이드

Part 1: Tool Use

1. Tool Use란?

Tool Use는 Claude가 직접 정의한 함수나 Anthropic이 제공하는 함수를 호출할 수 있는 기능입니다. Claude는 사용자 요청과 도구 설명을 바탕으로 언제 도구를 호출할지 결정하고, 애플리케이션이 실행하는 구조화된 호출을 반환합니다.

핵심을 딱 한 줄로 정리하면:

"Claude는 직접 코드를 실행하지 않는다. 대신 '이 함수를 이 인수로 실행해줘'라는 구조화된 지시를 반환하고, 실행은 여러분의 코드가 한다."

일반 Chat:

사용자: "오늘 서울 날씨는?"

Claude: "저는 실시간 정보가 없어서 모릅니다" ← 한계

Tool Use:

사용자: "오늘 서울 날씨는?"

Claude: `get_weather(city="서울")` 호출 요청

앱: 날씨 API 실제 호출 → 결과 반환

Claude: "서울은 현재 23°C, 맑음입니다" ✓

2. Tool Use가 Agent의 핵심인 이유

도구 접근은 에이전트에게 줄 수 있는 가장 높은 레버리지 기본 요소 중 하나입니다. LAB-Bench FigQA(과학 도표 해석)와 SWE-bench(실세계 소프트웨어 엔지니어링) 같은 벤치마크에서 기본 도구만 추가해도 성능이 크게 향상되어 인간 전문가 기준선을 넘는 경우가 많습니다.

3. Tool Use 동작 원리: 6단계 루프

Claude는 외부 코드를 직접 실행하지 않습니다. 대신 도구를 사용할 의도를 신호로 보내고 결과를 기다리며 그 결과를 바탕으로 최종 응답을 생성하는 다단계 대화에 참여합니다.

① 사용자 요청 + 도구 목록 전달

```
[ messages: [{"role": "user",  
  "content": "서울 날씨 알려줘"}],  
  tools: [get_weather 정의]
```

↓

② Claude: 도구 필요성 판단

"이 요청에는 **get_weather** 도구가 필요하다"

↓

③ Claude: tool_use 블록 반환 (stop_reason: "tool_use")

```
[ {"type": "tool_use",  
  "name": "get_weather",  
  "input": {"city": "서울"}}]
```

↓

④ 앱: 실제 날씨 API 호출 → 결과 획득

↓

⑤ tool_result를 Claude에 다시 전달

```
[ {"type": "tool_result",  
  "content": "23°C, 맑음, 습도 45%"}]
```

↓

⑥ Claude: 결과를 보고 최종 자연어 응답 생성

"서울은 현재 23°C로 맑은 날씨입니다."

4. Tool의 2가지 종류: Client vs Server

도구는 주로 코드가 실행되는 위치에 따라 구분됩니다. Client 도구(사용자 정의 도구, bash, text_editor 같은 Anthropic 스키마 도구 포함)는 여러분의 애플리케이션에서 실행됩니다. Claude가 stop_reason: "tool_use"와 하나 이상의 tool_use 블록으로 응답하면, 여러분의 코드가 작업을 실행하고 tool_result를 돌려줍니다. Server 도구의 경우 Anthropic이 실행을 처리합니다.

Client Tool (내가 실행):	Server Tool (Anthropic이 실행):
내 DB 조회 함수 내 API 호출 함수 내 파일 처리 함수	web_search (Anthropic 제공) code_execution (실행까지 Anthropic이 담당)
장점: 완전한 제어	장점: 설정 없이 즉시 사용
단점: 실행 코드 직접 작성 필요	단점: 추가 비용 발생 가능

5. Tool 정의 방법 (코드 예시)

python

```
import anthropic
```

```
client = anthropic.Anthropic()
```

```
# ① 도구 정의: 이름, 설명, 입력 스키마
```

```
tools = [  
    {  
        "name": "get_weather",  
        "description": "특정 도시의 현재 날씨를 가져옵니다. "  
            "날씨 관련 질문이 있을 때 사용하세요.",  
        "input_schema": {  
            "type": "object",  
            "properties": {  
                "city": {  
                    "type": "string",  
                    "description": "날씨를 조회할 도시명 (예: 서울, 부산)"  
                },  
                "unit": {  
                    "type": "string",  
                    "enum": ["celsius", "fahrenheit"],  
                    "description": "온도 단위"  
                }  
            },  
            "required": ["city"]  
        }  
    },  
    {  
        "name": "search_database",  
        "description": "내부 데이터베이스에서 고객 정보를 검색합니다.",  
        "input_schema": {  
            "type": "object",  
            "properties": {  
                "customer_id": {"type": "string"},  
                "fields": {  
                    "type": "array",  
                    "items": {"type": "string"}  
                }  
            },  
            "required": ["customer_id"]  
        }  
    }  
]
```

```

# ② API 호출
response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=1024,
    tools=tools,
    messages=[{"role": "user", "content": "서울 날씨 알려줘"}]
)

# ③ tool_use 블록 처리
for block in response.content:
    if block.type == "tool_use":
        tool_name = block.name
        tool_input = block.input

        # 실제 함수 실행 (앱 코드)
        if tool_name == "get_weather":
            result = call_weather_api(tool_input["city"])
        elif tool_name == "search_database":
            result = query_db(tool_input["customer_id"])

# ④ 결과를 Claude에 다시 전달
final_response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=1024,
    tools=tools,
    messages=[
        {"role": "user", "content": "서울 날씨 알려줘"},
        {"role": "assistant", "content": response.content},
        {
            "role": "user",
            "content": [{
                "type": "tool_result",
                "tool_use_id": block.id,
                "content": str(result)
            }]
        }
    ]
)

```

6. 고급 Tool Use: 병렬 호출

멀티 단계 워크플로우에서 Claude는 샘플링 없이 도구를 연속으로 또는 루프로 호출해 토큰과 레이턴시를 절약할 수 있습니다. 조건부 로직을 통해 중간 도구 결과에 따라 결정을 내릴 수 있습니다.

```
python
# Claude가 한 번에 여러 도구를 동시에 요청할 수 있음
response.content = [
    {"type": "tool_use", "name": "get_weather", "input": {"city": "서울"}},
    {"type": "tool_use", "name": "get_weather", "input": {"city": "부산"}},
    {"type": "tool_use", "name": "search_database", "input": {"customer_id": "123"}}
]
# → 세 가지를 병렬로 실행해 응답 속도 3배 향상
```

7. Anthropic 제공 Server Tools

Anthropic은 Tool Search Tool, Programmatic Tool Calling, 웹 검색 등의 서버 사이드 도구를 제공합니다. 에이전트가 컨텍스트 창을 미리 채우지 않고 수천 개의 도구에 온디맨드로 접근할 수 있게 해줍니다.

도구	설명	용도
<code>web_search</code>	실시간 웹 검색	최신 정보 조회
<code>code_execution</code>	코드 실행 샌드박스	계산, 데이터 분석
<code>bash</code>	셸 명령어 실행	파일 처리, 시스템 명령
<code>text_editor</code>	텍스트 파일 편집	코드/문서 수정
<code>tool_search</code>	도구 동적 탐색	수천 개 MCP 도구 중 필요한 것만 로드

8. Tool 성능 최적화 팁

Tool 정의를 최적화하는 것이 가장 중요합니다. Description 필드는 무엇을 하는지, 언제 사용해야 하는지, 사용하지 말아야 할 때는 언제인지를 명확히

설명해야 합니다. 여러 도구가 유사한 작업을 수행한다면 지나치게 일반적인 이름은 피하고 직관적이고 구분되는 이름을 사용하세요.

```
python
# ❌ 나쁜 Tool 정의
{
    "name": "do_stuff",
    "description": "무언가를 합니다",
    ...
}

# ✅ 좋은 Tool 정의
{
    "name": "get_customer_orders",
    "description": "특정 고객의 주문 이력을 조회합니다. "
                  "고객 서비스 관련 질문이나 주문 상태 확인 요청 시 사용하세요. "
                  "단순 제품 정보 조회에는 사용하지 마세요.",
    ...
}
```

Part 2: Computer Use

9. Computer Use란?

Claude Code Computer Use는 AI가 마우스, 키보드, 화면을 제어해 인간이 하는 방식으로 작업을 완료하는 기능입니다. 지시를 생성하는 대신 Claude가 직접 브라우저를 열고 페이지를 탐색하고 양식을 작성하고 워크플로우를 클릭하며 결과를 확인할 수 있습니다.

Tool Use와의 근본적 차이:

Tool Use:

함수 → 구조화된 결과

"날씨 API 호출해줘"
→ JSON 결과 반환

API가 있는 모든 것에 사용
빠름 (ms 단위)

Computer Use:

스크린샷 → 시각적 판단 → 마우스/키보드

"브라우저 열고 날씨 사이트 접속해서
결과 스크린샷 찍어서 알려줘"

API가 없는 모든 것에 사용
느림 (초~분 단위)

10. Computer Use 동작 원리: 비전 피드백 루프

Computer Use의 핵심은 스크린샷-분석-액션 피드백 루프입니다. Claude가 화면을 캡처하고, 이미지를 분석해 픽셀 수준에서 UI 요소를 파악하고, 다음 행동을 결정하고, 액션을 실행한 후 결과 검증을 위해 다시 스크린샷을 찍습니다. 이 사이클이 작업 완료까지 반복됩니다.

① 화면 스크린샷 캡처



② Claude 비전 모델로 화면 분석

"로그인 버튼이 (450, 320) 위치에 있다"



③ 액션 결정 및 실행

`click(x=450, y=320)`



④ 새 스크린샷으로 결과 확인

"로그인 성공, 대시보드 화면으로 이동됨"



⑤ 다음 단계로 진행 또는 오류 시 재시도



[작업 완료까지 반복]

이 아키텍처는 클라이언트 사이드입니다. Claude가 OS를 직접 건드리지 않습니다. 로컬 런타임이 스크린샷을 캡처하고 API로 전송하면 Claude가 좌표와 키스트로크가 담긴 tool-use 요청을 반환하고, 그것을 macOS Accessibility API를

통해 로컬에서 실행한 뒤 새 스크린샷으로 루프가 이어집니다.

11. 지원하는 모든 액션

Computer Use가 지원하는 입력 유형은 마우스 액션(이동, 클릭, 더블클릭, 드래그), 키보드 입력(텍스트 타이핑, 키 조합 — Ctrl+C, Cmd+Tab 등), 스크롤(방향별 수직·수평), 스크린샷 캡처입니다.

마우스 액션:

<code>click(x, y)</code>	← 좌클릭
<code>right_click(x, y)</code>	← 우클릭
<code>double_click(x, y)</code>	← 더블클릭
<code>drag(start, end)</code>	← 드래그
<code>left_mouse_down/up</code>	← 세밀한 마우스 제어
<code>scroll(x, y, direction)</code>	← 스크롤

키보드 액션:

<code>type("텍스트")</code>	← 텍스트 입력
<code>key("ctrl+c")</code>	← 키 조합
<code>key("enter")</code>	← 단일 키
<code>hold_key("shift")</code>	← 키 홀드

고급 액션:

<code>screenshot()</code>	← 현재 화면 캡처
<code>zoom(region)</code>	← 특정 영역 확대 (세부 내용 읽기)
<code>wait(seconds)</code>	← 로딩 대기

12. 3가지 접근 경로 비교

Computer Use는 현재 두 가지 다른 계약을 가리킵니다. 하나는 자체 샌드박스 내에서 액션을 실행하는 빌더를 위한 Anthropic API 도구이고, 다른 하나는 자신의 실제 컴퓨터에서 Claude가 작동하길 원하는 사람들을 위한 Cowork 또는 Claude Code의 데스크톱 워크플로우입니다.

구분	API (개발자용)	Claude Code (개발자)	Cowork (일반 사용자)
환경	직접 구성한 VM/샌드박스	실제 로컬 데스크톱	실제 로컬 데스크톱
제어권	완전히 내가 소유	Claude Code가 관리	Cowork가 관리
대상	제품 빌더	개발자	비개발자
설정	직접 루프 구현	v2.1.85+ 설치 후 활성화	데스크톱 앱 설치
요금	API 사용량	Pro/Max 플랜	Pro/Max 플랜

13. 실전 활용 예시

예시 1: 네이티브 앱 빌드 & 검증 Claude Code로 macOS 메뉴바 앱을 만들어달라고 하면, Claude가 Swift 코드를 작성하고 컴파일하고 앱을 실행하고 모든 컨트롤을 직접 클릭해 여러분이 열어보기 전에 작동하는지 검증합니다.

```
"macOS 메뉴바 앱 만들어줘"
  → Swift 코드 작성 (Bash 도구)
  → xcodebuild로 컴파일 (Bash 도구)
  → 앱 실행 (Computer Use)
  → 메뉴바 아이콘 클릭 (Computer Use)
  → 각 기능 동작 확인 (Computer Use + 스크린샷)
  → 버그 발견 시 코드 수정 후 재검증
```

예시 2: 시각적 버그 수정 좁은 창에서 모달이 잘린다고 하면, Claude가 앱 창 크기를 조절해 버그를 재현하고 잘린 상태를 스크린샷으로 찍은 뒤 관련 CSS를 확인합니다.

"좁은 창에서 모달 잘리는 버그 수정해줘"

- 앱 창 크기 축소 (Computer Use)
- 버그 상태 스크린샷 (Computer Use)
- CSS 파일 분석 (Bash 도구)
- 수정 적용 (Edit 도구)
- 수정 후 재확인 (Computer Use)

예시 3: API 없는 레거시 시스템 ERP, 정부 포털, API 없는 내부 도구. Claude가 순수 화면 제어로 이런 인터페이스를 탐색할 수 있습니다. 데이터 입력, 양식 작성, 보고서 추출 — API 없는 레거시 시스템을 처리하는 팀에게 이것만으로도 충분한 가치가 있습니다.

"이 레거시 ERP에서 이번 달 재고 데이터 추출해줘"

- ERP 프로그램 실행 (Computer Use)
- 로그인 (Computer Use - 키보드)
- 재고 메뉴 탐색 (Computer Use - 클릭)
- 날짜 필터 설정 (Computer Use)
- 보고서 내보내기 클릭 (Computer Use)
- 다운로드된 파일 확인 (Bash 도구)

14. 앱 카테고리별 접근 권한

브라우저(Safari, Chrome 등)는 읽기 전용으로, Claude가 화면을 볼 수는 있지만 클릭, 타이핑, 탐색은 불가능합니다. 브라우저 상호작용에는 Chrome 확장이 필요합니다. 터미널과 IDE(Terminal, iTerm, VS Code 등)는 클릭 전용으로, 버튼 클릭과 스크롤만 가능하며 타이핑이나 키보드 단축키는 불가능합니다. 셸 명령에는 Bash 도구를 사용하세요. 그 외 모든 앱은 완전한 제어가 가능합니다.

앱 카테고리별 허용 수준:

브라우저 (Chrome, Safari):

✅ 스크린샷 (보기)

❌ 클릭, 타이핑 불가

→ 브라우저 자동화는 Claude in Chrome 사용

터미널/IDE (Terminal, VS Code):

✅ 버튼 클릭, 스크롤

❌ 타이핑, 키보드 단축키 불가

→ 명령 실행은 Bash 도구 사용

그 외 모든 앱 (Figma, Excel, ERPs 등):

✅ 완전한 마우스 + 키보드 제어

15. Computer Use 보안 주의사항

Computer Use는 클라이언트 사이드 도구입니다. 세션의 모든 스크린샷, 마우스 액션, 키보드 입력 및 관련 파일은 Anthropic이 아닌 여러분의 환경에 캡처·저장됩니다.

Claude가 화면을 제어하기 위해 지속적으로 스크린샷을 찍기 때문에 화면에 보이는 모든 정보(개인 문서, 개인 메시지, 기밀 데이터 포함)를 잠재적으로 볼 수 있습니다. Anthropic은 사용 전에 민감한 애플리케이션을 닫을 것을 명시적으로 권장합니다.

✅ 권장 사항:

- VM/컨테이너 격리 환경에서 실행
- 민감한 앱은 미리 닫기
- 모든 세션 로그 기록
- 실제 비밀번호 대신 테스트 계정 사용
- 중요 결정 전 **Human-in-the-Loop** 적용

❌ 주의 사항:

- 금융/투자 플랫폼 → 기본 차단
- **Admin** 권한 계정으로 실행 금지
- 무감독 장시간 실행 지양
- 프롬프트 인젝션 주의 (화면의 악성 텍스트)

16. Tool Use vs Computer Use 최종 비교

Claude가 앱이나 서비스와 상호작용하는 방법에는 여러 가지가 있습니다. Computer Use는 가장 광범위하고 느리기 때문에 Claude는 가장 정밀한 도구를 먼저 시도합니다. 서비스의 MCP 서버가 있으면 그것을 사용하고, 셸 명령이면 Bash를 사용하며, 브라우저 작업이고 Claude in Chrome이 설정되어 있으면 그것을 사용하고, 이 중 아무것도 해당되지 않으면 Computer Use를 사용합니다.

Claude의 도구 선택 우선순위:

- 1순위: **MCP Connector** → API가 있는 서비스 (가장 빠름)
- 2순위: **Bash Tool** → 명령줄로 가능한 작업
- 3순위: **Claude in Chrome** → 브라우저 자동화
- 4순위: **Computer Use** → 위 어떤 것도 해당 없을 때 (가장 느림)

항목	Tool Use	Computer Use
작동 방식	함수 호출 → JSON 결과	스크린샷 → 시각 분석 → 액션
속도	밀리초	초~분
정확도	매우 높음	중간 (UI 변화에 취약)
필요 조건	API/함수 정의	없음 (화면만 있으면 됨)
주요 용도	DB 조회, API 호출, 계산	레거시 앱, GUI 전용 툴
비용	낮음	높음 (스크린샷 처리)
보안	높음	주의 필요

다음으로 **Memory 시스템** (단기/장기/벡터 메모리), **Hooks 시스템**, **Claude in Chrome**, 또는 **RAG와 Agent의 결합 패턴**을 더 깊이 알아볼까요? 🚀

Claude Memory 시스템 완벽 가이드

1. Memory 시스템이 필요한 이유

Claude의 근본적인 한계를 먼저 이해해야 합니다.

매 세션마다 기억 초기화 (기본 상태):

월요일 세션:

사용자: "우리 프로젝트는 **FastAPI + PostgreSQL**이야"

Claude: 작업 수행... 세션 종료

↓ (기억 사라짐)

화요일 세션:

사용자: "어제 말한 것처럼..."

Claude: "저는 이전 대화를 기억하지 못합니다" ← 매번 반복

각 Claude Code 세션은 새로운 컨텍스트 창으로 시작합니다. 두 가지 메커니즘이 세션 간 지식을 전달합니다: 지속적인 컨텍스트를 제공하기 위해 직접 작성하는 CLAUDE.md 파일과, 수정 사항 및 선호도를 기반으로 Claude가 자율적으로 메모를 쌓는 Auto Memory입니다.

2. Memory의 4가지 계층

인지과학의 인간 기억 모델에서 영감을 받은 구조입니다.

Claude Memory 4계층

- | | |
|---------------------------|------------------|
| ① In-Context (작업 기억) | ← 현재 대화 = RAM |
| ② CLAUDE.md (절차 기억) | ← 내가 쓴 지시 = 매뉴얼 |
| ③ Auto Memory (일화 기억) | ← Claude가 자동 학습 |
| ④ External Memory (외부 기억) | ← 벡터DB, 파일 = 도서관 |

3. 계층 ①: In-Context Memory (작업 기억)

현재 대화의 컨텍스트 창 자체가 단기 메모리입니다.

컨텍스트 창 = 현재 세션의 모든 것:

	System Prompt	
	+ CLAUDE.md 내용	
	+ Auto Memory 내용	
	+ 현재 대화 전체 (질문/답변/도구 결과)	
	+ 업로드된 파일	
	한계: Claude Opus 4.6 = 1M 토큰	
	Claude Sonnet 4.6 = 200K 토큰	

↓ 세션 종료 시 전부 사라짐

컨텍스트 관리 전략:

컨텍스트 편집은 클라이언트 사이드에서 특정 도구 결과를 제거하고, Compaction은 컨텍스트 창 한계에 가까워지면 서버 사이드에서 전체 대화를 자동으로 요약합니다. 장기 에이전틱 워크플로우에는 두 가지를 함께 사용하는 것을 고려하세요. Compaction은 클라이언트 사이드 관리 없이 활성 컨텍스트를 관리 가능하게 유지하고, Memory는 Compaction 경계를 넘어 중요 정보를 보존해 요약에서 핵심이 사라지지 않도록 합니다.

4. 계층 ②: CLAUDE.md (절차 기억 — 내가 직접 작성)

CLAUDE.md 파일은 프로젝트, 개인 워크플로우, 전체 조직에 Claude에게 지속적인 지시사항을 주는 마크다운 파일입니다. 이 파일들을 일반 텍스트로 작성하면 Claude가 매 세션 시작 시 읽습니다. 새 팀원이 생산적으로 일하기 위해 같은 컨텍스트가 필요할 때 다시 설명해야 하는 내용을 적어두는 곳으로 생각하세요.

저장 위치별 범위 (우선순위 순)

우선순위 (높음 → 낮음):

- | | |
|--------------------------------|---------------------|
| 1. /project/frontend/CLAUDE.md | ← 서브디렉토리 전용 |
| 2. /project/CLAUDE.md | ← 프로젝트 전체 |
| 3. ~/.claude/CLAUDE.md | ← 내 모든 프로젝트 |
| 4. 조직 관리 CLAUDE.md | ← 팀 전체 (Enterprise) |

CLAUDE.md 작성 예시 (실전)

markdown

MyApp 프로젝트 가이드

기술 스택

- Backend: FastAPI 0.115 + PostgreSQL 16
- Frontend: React 18 + TypeScript 5 + Tailwind
- 테스트: pytest (백엔드), Vitest (프론트엔드)
- 배포: Docker + GitHub Actions

자주 쓰는 명령어

- 개발 서버 시작: ``make dev``
- 테스트 실행: ``make test``
- DB 마이그레이션: ``alembic upgrade head``
- 린트: ``make lint``

코딩 컨벤션

- Python: PEP 8 준수, 타입 힌트 필수
- 커밋: Conventional Commits 형식
- API 엔드포인트: RESTful, snake_case

절대 하지 말 것

- main 브랜치에 직접 푸시 금지
- .env 파일 커밋 금지
- print() 디버깅 코드 커밋 금지

아키텍처 결정 사항

- 인증: JWT (access 15분 + refresh 7일)
- 캐싱: Redis (세션/자주 조회 데이터)
- 파일 업로드: AWS S3 presigned URL 방식

경로별 범위 지정 (고급)

markdown

<!-- 특정 디렉토리에만 적용되는 규칙 -->

[src/api/**]

API 엔드포인트 작성 시 항상 **OpenAPI** 문서 업데이트

[src/models/**]

모델 변경 시 반드시 마이그레이션 파일 생성

[tests/**]

테스트 커버리지 **90%** 이상 유지

5. 계층 ③: Auto Memory (일화 기억 — Claude가 자동 작성)

Auto Memory란?

Auto Memory는 개발자가 수동으로 CLAUDE.md를 업데이트하지 않고도 에이전트가 세션을 거치면서 프로젝트별 지식을 축적할 수 있게 해주는 기능입니다. 수동 문서화를 기다리는 대신 Claude Code가 세션 중에 배운 유용한 것들을 파악하고 CLAUDE.md에 자율적으로 기록합니다.

Auto Memory 동작 흐름:

세션 중:

사용자: "그 명령어 아니야, `npm run dev:local`이야"

↓

Claude: "수정 내용 학습: dev 서버 명령어 = npm run dev:local"

↓

세션 종료 시:

Claude → CLAUDE.md에 자동으로 기록:

빌드 명령어

- 개발 서버: npm run dev:local (npm start 아님)"

↓

다음 세션:

Claude: (처음부터 npm run dev:local 사용) ✓

Claude가 자동으로 기억하는 것

Auto Memory는 Claude가 작업하면서 스스로 메모를 저장합니다: 빌드 명령어, 디버깅 인사이트, 아키텍처 메모, 코드 스타일 선호도, 워크플로우 습관. Claude는 매 세션마다 저장하지 않습니다. 미래 대화에서 유용할지 여부를 기반으로 기억할 가치가 있는지 결정합니다.

✓ 자동으로 기억하는 것:

- 수정받은 명령어/경로
- 반복되는 코드 패턴
- 아키텍처 결정 사항
- 팀 컨벤션
- 자주 발생하는 에러 해결법

✗ 기억하지 않는 것:

- 일회성 답변
- 임시 디버깅 로그
- 단순 사실 질문 답변
- 너무 상황 의존적인 것

Auto Memory 관리 (/memory 명령어)

```
bash
```

```
# 현재 메모리 상태 확인 + 수정
```

```
/memory
```

```
# 열리는 인터페이스:
```

```
| Auto Memory [켜기/끄기 토글] |
```

```
| 현재 저장된 메모리: |
```

- npm run dev:local (dev 서버)
- PostgreSQL 16 사용 중
- JWT 인증, refresh 7일

```
| [항목 삭제] [전체 초기화] [수동 추가] |
```

6. Auto Dream: 메모리 자동 정제 시스템

AutoDream은 세션 사이에 Claude Code의 메모리를 통합하는 백그라운드 서브에이전트입니다. 이름은 의도적입니다. REM 수면 중에 이루어지는 생물학적 기억 통합 방식을 반영하며, 유희 시간에 실행되어 정확하고 관련성 있는 것만 남깁니다.

Auto Dream의 4단계 작업

Phase 1: 스캔 (현재 메모리 상태 파악)

MEMORY.md 인덱스 읽기

→ "현재 무엇을 알고 있나?"

Phase 2: 수집 (세션 기록에서 학습 내용 추출)

JSONL 세션 파일 검색:

- 사용자 수정 사항
- **"remember this"** 명시적 저장 요청
- 반복 패턴
- 아키텍처 결정

Phase 3: 통합 (중복 제거 + 정제)

핵심 단계입니다. Claude가 새 정보를 기존 토픽 파일에 병합하고 중요한 유지관리를 수행합니다: 상대적 날짜를 절대 날짜로 변환합니다. "어제 Redis를 쓰기로 했다"가 "2026-03-15에 Redis 사용 결정"이 됩니다. 모순된 사실을 삭제합니다. 3주 전에 Express에서 Fastify로 전환했다면 옛날 "API가 Express 사용" 항목이 제거됩니다. 오래된 메모리를 제거하고 겹치는 항목을 통합합니다. 세 개의 다른 세션이 같은 빌드 명령어 특이사항을 기록했다면 하나의 깔끔한 항목으로 통합됩니다.

Phase 4: 인덱스 최적화

MEMORY.md를 200줄 이내로 유지

(세션 시작 시 로드되는 한계)

→ 상세 내용은 토픽별 파일로 분리

topic-auth.md, topic-db.md, topic-devops.md

7. 계층 ④: External Memory (외부 기억)

Memory Tool (API 레벨)

활성화되면 Claude가 작업을 시작하기 전에 자동으로 메모리 디렉토리를 확인합니다. Claude는 /memories 디렉토리의 파일을 생성, 읽기, 업데이트, 삭제해서 작업하면서 배운 것을 저장하고 미래 대화에서 참조합니다. 이것은 클라이언트 사이드 도구이므로 Claude가 메모리 작업을 수행하는 도구 호출을

하면 여러분의 애플리케이션이 로컬에서 실행합니다. 메모리가 저장되는 위치와 방식을 완전히 제어할 수 있습니다.

```
python

# API에서 Memory Tool 활성화
import anthropic

client = anthropic.Anthropic()

response = client.messages.create(
    model="claude-opus-4-7",
    max_tokens=2048,
    tools=[
        {
            "type": "memory_20250124",    # Memory Tool 활성화
            "name": "memory"
        }
    ],
    messages=[{
        "role": "user",
        "content": "이전에 논의한 아키텍처 결정사항 확인해줘"
    }]
)

# Claude가 자동으로 /memories 디렉토리 먼저 확인 후 작업
```

Memory Tool 자동 프로토콜

Memory Tool이 활성화되면 시스템 프롬프트에 다음 지시사항이 자동 포함됩니다: "중요: 다른 어떤 것도 하기 전에 항상 메모리 디렉토리를 먼저 확인하세요. 메모리 프로토콜: 1. 이전 진행 상황을 확인하기 위해 memory 도구의 view 명령을 사용하세요. 2. 작업을 진행하면서 상태/진행사항/생각 등을 메모리에 기록하세요. 컨텍스트 창이 언제든지 초기화될 수 있다고 가정하세요. 기록되지 않은 진행 상황은 잃어버릴 수 있습니다."

Memory Tool 자동 동작:

세션 시작



`/memories` 디렉토리 자동 스캔



관련 메모리 파일 로드



작업 수행 중...



중요 정보 발견 시 `/memories`에 자동 저장



세션 종료 → 다음 세션에서 계속

Vector DB를 이용한 장기 외부 메모리

대규모 지식 베이스가 필요한 경우 RAG 패턴으로 외부 메모리를 구현합니다.

python

```
from anthropic import Anthropic
import chromadb # 벡터 DB

client = Anthropic()
vector_db = chromadb.Client()
collection = vector_db.create_collection("long_term_memory")

def store_memory(content: str, metadata: dict):
    """중요한 정보를 벡터 DB에 저장"""
    collection.add(
        documents=[content],
        metadatas=[metadata],
        ids=[f"mem_{hash(content)}"]
    )

def retrieve_memory(query: str, n_results=5) -> str:
    """쿼리와 의미적으로 유사한 메모리 검색"""
    results = collection.query(
        query_texts=[query],
        n_results=n_results
    )
    return "\n".join(results["documents"][0])

def chat_with_memory(user_message: str):
    # 1. 관련 과거 메모리 검색
    relevant_memories = retrieve_memory(user_message)

    # 2. 메모리를 컨텍스트로 주입
    system = f"""당신은 사용자를 도와주는 어시스턴트입니다.
```

관련 과거 컨텍스트:

```
{relevant_memories}
```

이 정보를 참고해 일관성 있게 답변하세요."""

```
# 3. Claude 호출
```

```
response = client.messages.create(  
    model="claude-sonnet-4-6",  
    max_tokens=1024,  
    system=system,  
    messages=[{"role": "user", "content": user_message}]  
)
```

```
# 4. 중요한 새 정보는 메모리에 저장
```

```
store_memory(  
    content=f"Q: {user_message}\nA: {response.content[0].text}",  
    metadata={"type": "conversation", "timestamp": "2026-05-09"}  
)
```

```
return response.content[0].text
```

8. Claude.ai 웹에서의 Memory

claude.ai 웹/앱 인터페이스에서 제공하는 Memory 기능입니다.

작동 방식

Memory 관리는 LLM의 컨텍스트 창 내부와 외부 저장소에서 정보를 관리하는 방법과, 이 두 곳 간에 정보를 전달하는 방법을 포함합니다. 핵심 구성 요소는 생성, 저장, 검색, 통합, 업데이트, 삭제(망각)입니다.

사용자와의 대화에서 자동 추출:

"저는 시니어 백엔드 개발자입니다" → 메모리에 저장

"Python을 주로 씁니다" → 메모리에 저장

"우리 팀은 Git Flow를 씁니다" → 메모리에 저장

다음 대화에서 자동 활용:

Claude: (파이썬 코드 예시를 기본으로 제공)

Claude: (시니어 수준의 설명 제공)

Claude: (Git Flow 기준으로 브랜치 전략 설명)

Settings에서 Memory 관리

Settings → Memory 에서:

Memory 설정
☒ Memory 활성화 (토글)
저장된 메모리:
• 시니어 백엔드 개발자 • Python/FastAPI 사용 • Git Flow 브랜치 전략
[개별 삭제] [전체 삭제] [수동 추가]

수동으로 메모리 추가/제어

대화 중 명시적 지시:

"이건 기억해줘: 우리 DB 이름은 prod_db야"

"앞으로 항상 한국어로 답해줘"

"이건 잊어버려: 내 직책 정보"

"내 기술 스택에 Go를 추가해줘"

9. 4가지 Memory 비교: 언제 무엇을 쓰나

구분	In-Context	CLAUDE.md	Auto Memory	External (Vector DB)
지속성	세션 내만	영구 (파일)	영구 (자동)	영구 (DB)
용량	컨텍스트 창 한계	수백 줄 권장	자동 압축	무제한
입력 방식	자동 (대화)	직접 작성	Claude 자동	코드로 관리
검색 방식	없음 (전부 로드)	없음 (전부 로드)	없음 (전부 로드)	의미 기반 검색
최적 용도	현재 작업	고정 규칙/컨벤션	동적 학습	대규모 지식 베이스
사용 환경	어디서나	Claude Code	Claude Code	API 직접 구현

10. 메모리 계층 전체 흐름도

세션 시작



컨텍스트 창에 로드 (In-Context)

[CLAUDE.md 전체]

+ [Auto Memory 내용]

+ [External Memory 검색 결과]

= 세션 시작 기준 지식



작업 진행 (In-Context 확장)

+ 사용자 메시지

+ Claude 응답

+ Tool Use 결과

+ 업로드 파일



세션 종료 시 정제 및 저장

중요 인사이트 → Auto Memory에 저장

수동 지시사항 → CLAUDE.md 업데이트

중요 사실 → External Memory에 저장



Auto Dream (백그라운드 정제)

5+ 세션 & 24시간+ 경과 시 자동 실행

중복 제거 + 모순 해결 + 날짜 정규화

MEMORY.md 200줄 이내로 최적화

11. 실전 Memory 전략: 용도별 최적 구성

개인 개발자 설정

markdown

~/.claude/CLAUDE.md (전역 설정)

내 개발 스타일

- 언어: Python 선호, 타입 힌트 필수
- 에디터: VS Code, vim 단축키
- 터미널: zsh + oh-my-zsh
- 설명 방식: 예제 먼저, 이론 나중에
- 언어: 한국어로 답변

팀 프로젝트 설정

markdown

/project/CLAUDE.md (프로젝트 공유)

프로젝트: MyApp

- 스택: FastAPI + React + PostgreSQL
- 테스트: pytest, 커버리지 90% 이상
- CI/CD: GitHub Actions
- 브랜치: feature/* → develop → main

팀 컨벤션

- PR 크기: 300줄 이하
- 리뷰어: 최소 2명 승인 필요
- 마이그레이션: 항상 롤백 포함

장기 프로젝트 (Memory Tool 활용)

여러 에이전트 세션에 걸친 장기 소프트웨어 프로젝트의 경우, 메모리 파일을 즉흥적으로 작성하는 것이 아니라 의도적으로 부트스트랩해야 합니다. 초기화 세션에서 실질적인 작업이 시작되기 전에 메모리 아티팩트를 설정합니다: 진행 로그(완료된 것과 다음에 올 것 추적), 기능 체크리스트(작업 범위 정의), 참조

아키텍처 문서가 포함됩니다.

```
/memories/  
├─ MEMORY.md          ← 인덱스 (200줄 이하)  
├─ progress.md        ← 진행 상황 추적  
├─ decisions.md       ← 아키텍처 결정 기록  
├─ topic-auth.md      ← 인증 관련 메모리  
├─ topic-db.md        ← DB 관련 메모리  
└─ topic-devops.md    ← DevOps 관련 메모리
```

12. 보안 및 프라이버시 고려사항

프로젝트 문서화는 안정적인 결론에 적합하고 메모리 시스템은 대화 중 생성된 컨텍스트에 더 적합합니다. 둘은 서로 대체제가 아닙니다. 안정적인 지식은 프로젝트 문서에 작성하고 메모리 시스템은 대화 컨텍스트 검색을 돕는 데 사용하는 것이 좋은 워크플로우입니다.

✅ 메모리에 저장해도 되는 것:

- 기술 스택 / 아키텍처 결정
- 코딩 컨벤션 / 팀 규칙
- 빌드 명령어 / 워크플로우
- 반복되는 에러 해결법

❌ 절대 메모리에 저장하면 안 되는 것:

- API 키 / 비밀번호
- 개인 식별 정보 (PII)
- 고객 데이터
- 내부 보안 정책

<private> 태그로 메모리 저장 방지:

<private>

DB 비밀번호: super_secret_123

</private>

다음으로 RAG (검색 증강 생성) 심화, Hooks 시스템, Claude Code 실전 워크플로우 구축, 또는 지금까지 배운 모든 개념의 통합 실습으로 이어가볼까요?



Claude Code Hooks 시스템 완벽 가이드

1. Hooks란?

Hooks는 Claude Code의 라이프사이클에서 특정 지점에 자동으로 실행되는 사용자 정의 셸 명령어, HTTP 엔드포인트, 또는 LLM 프롬프트입니다.

CLAUDE.md와의 결정적 차이:

CLAUDE.md는 제안입니다. Claude Code는 보통 따르지만 보장되지 않습니다. Hooks는 결정론적(deterministic)입니다 — 항상 실행됩니다. CLAUDE.md는 가이드라인에 사용하고("npm보다 Bun 선호"), Hooks는 절대 어겨서는 안 되는 규칙에 사용하세요("항상 Prettier로 포맷", ".env 파일 절대 수정 금지").

CLAUDE.md: "테스트 항상 실행해줘"

→ Claude가 가끔 잊을 수 있음 ⚠️

Hooks:

PostToolUse → 파일 수정 시 자동으로 테스트 실행

→ 예외 없이 100% 보장 ✅

2. Hooks의 핵심 구조: 3가지 구성 요소

공식 문서는 세 가지 수준의 정확한 용어를 사용합니다: Hook

Event(라이프사이클 지점), Matcher Group(필터), Hook Handler(실행되는 셸 명령어, HTTP 엔드포인트, MCP 도구, 프롬프트, 또는 에이전트)입니다.

Hook 실행 흐름:

[Event 발생]

↓

[Matcher: 이 Hook이 관련 있나?] → No → 통과

↓ Yes

[Handler 실행]

↓

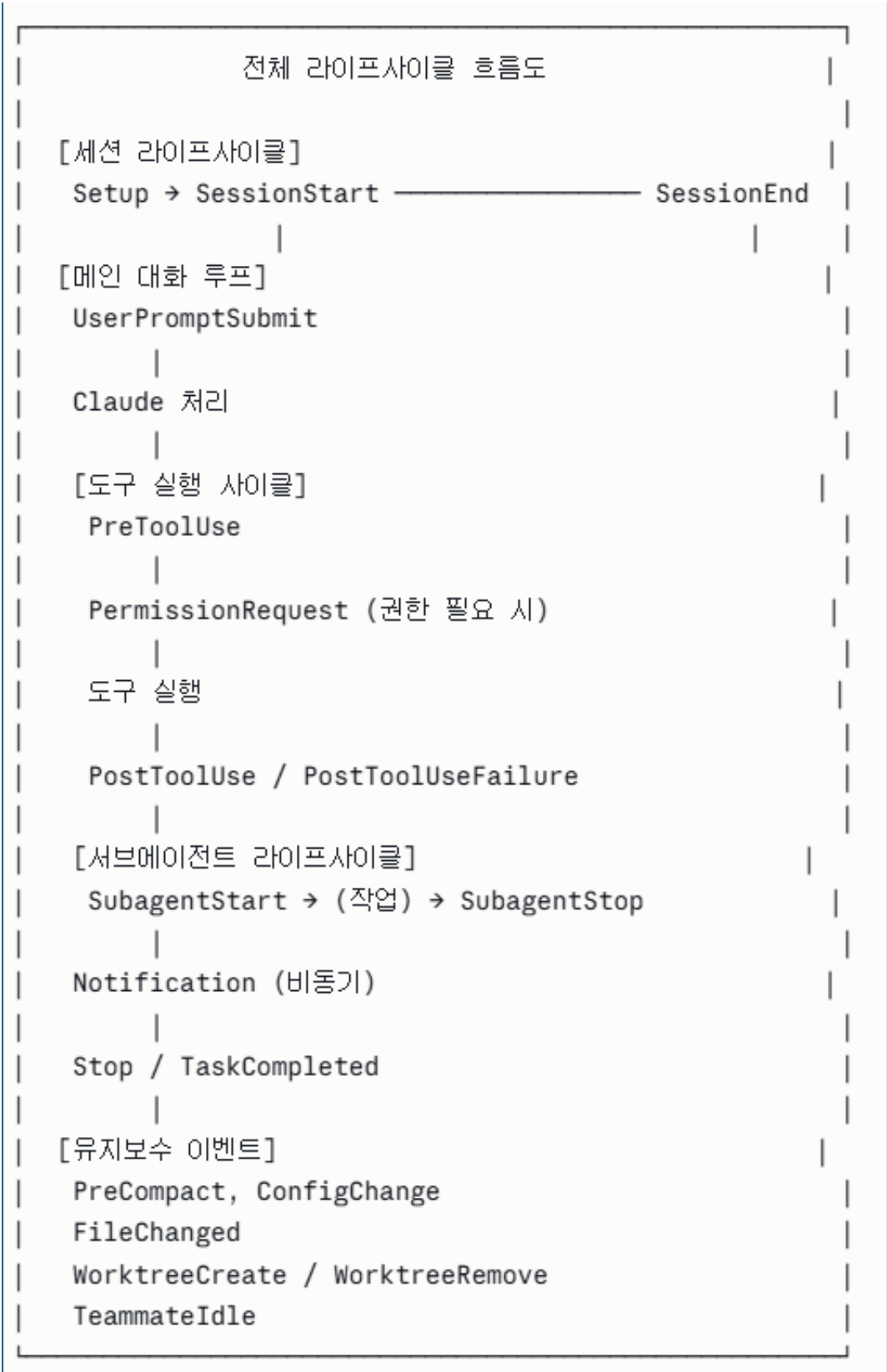
[Exit Code / JSON Output으로 결과 반환]

↓

[Claude 계속 진행 or 차단]

3. 전체 라이프사이클 이벤트 (17가지)

Claude Code는 17가지의 고유한 라이프사이클 이벤트를 노출합니다. 각 이벤트는 Claude 작업의 특정 순간에 발생하여 커스텀 로직을 삽입할 정확한 지점을 제공합니다.



이벤트별 상세 설명

이벤트	발생 시점	차단 가능	매치 지원
SessionStart	세션 시작/재개/clear 시	✗	✓
SessionEnd	세션 종료/에러 시	✗	✓
UserPromptSubmit	프롬프트 제출 직후	✓	✗
PreToolUse	도구 실행 직전	✓ (핵심)	✓
PermissionRequest	권한 요청 시	✓	✓
PostToolUse	도구 성공 완료 후	✗	✓
PostToolUseFailure	도구 실패 후	✗	✓
SubagentStart	서브에이전트 생성 시	✗	✓
SubagentStop	서브에이전트 완료 시	✓	✓
Notification	Claude 알림 전송 시	✗	✗
Stop	Claude 응답 완료 시	✓	✗
TaskCompleted	태스크 완료 시	✗	✗
PreCompact	컨텍스트 압축 직전	✗	✓
FileChanged	파일 변경 감지 시	✗	✓
ConfigChange	설정 파일 변경 시	✗	✓
WorktreeCreate	git worktree 생성 시	✗	✗
WorktreeRemove	git worktree 제거 시	✗	✗

4. Exit Code: Hook의 제어 메커니즘

Hook이 충돌하거나 0도 2도 아닌 exit code를 반환하면 Claude Code는 이를 hook 오류로 처리합니다. PreToolUse의 경우 오류가 발생한 hook은 일반적으로

도구 호출을 차단하지 않습니다(fail-open). PostToolUse의 경우 오류는 기록되지만 Claude의 워크플로우에 영향을 미치지 않습니다.

```
bash
```

```
exit 0    # ✅ 성공 - Claude 계속 진행
exit 1    # ⚠️ 오류 - 오류 메시지 표시, 기본적으로 계속 진행
exit 2    # ❌ 차단 - Claude 해당 액션 중단 (PreToolUse만 해당)
```

5. 4가지 Handler 타입

● Command Hook (가장 많이 사용, 90%)

Command Hook은 셸 명령어를 실행합니다. stdin으로 JSON을 받고 exit code로 결과를 반환합니다.

```
json
{
  "type": "command",
  "command": "./scripts/my-check.sh",
  "timeout": 10
}
```

● HTTP Hook (원격 검증, Feb 2026 추가)

HTTP Hook은 hook 이벤트를 웹 서버로 전송합니다. 팀 전체 정책을 적용하는 원격 검증 서비스, 중앙 로깅 등 이전에는 어렵거나 불가능했던 사용 사례를 열어줍니다.

```
json
{
  "type": "http",
  "url": "http://localhost:8080/hooks/pre-tool-use",
  "timeout": 30,
  "headers": {
    "Authorization": "Bearer $MY_TOKEN"
  },
  "allowedEnvVars": ["MY_TOKEN"]
}
```

● Prompt Hook (AI 판단)

LLM이 허용/차단 여부를 판단합니다. 규칙으로 표현하기 어려운 복잡한 결정에 사용합니다.

```
json
{
  "type": "prompt",
  "prompt": "다음 Bash 명령어가 안전한지 평가해줘: $ARGUMENTS\n데이터 손실이나 보안 위험이 있도"
}
```

```
{
  "type": "prompt",
  "prompt": "다음 Bash 명령어가 안전한지 평가해줘: $ARGUMENTS\n데이터 손실이나 보안 위험이 있으면 BLOCK, 안전하면 ALLOW를 반환해줘."
}
```

● Agent Hook (서브에이전트 검증)

Agent Hook은 Read, Grep, Glob 같은 도구 접근 권한을 가진 서브에이전트를 생성해 깊은 검증을 수행합니다.

json

```
{  
  "type": "agent",  
  "agent": "security-reviewer",  
  "prompt": "이 파일 변경사항의 보안 취약점을 분석해줘"  
}
```

```
{  
  "type": "agent",  
  "agent": "security-reviewer",  
  "prompt": "이 파일 변경사항의 보안 취약점을 분석해줘"  
}
```

6. 설정 파일 구조 및 범위

우선순위 (높음 → 낮음):

<code>.claude/settings.local.json</code>	← 개인용, gitignore (로컬만)
<code>.claude/settings.json</code>	← 프로젝트 공유 (git에 포함)
<code>~/.claude/settings.json</code>	← 내 모든 프로젝트
조직 관리 정책 (Enterprise)	← 팀 전체 강제 적용

json

```
// .claude/settings.json 기본 구조
{
  "hooks": {
    "이벤트명": [
      {
        "matcher": "매치 패턴",
        "hooks": [
          {
            "type": "command",
            "command": "실행할 명령어",
            "timeout": 10
          }
        ]
      }
    ]
  }
}
```

7. 실전 Hook 레시피 20가지

● 자동화 (PostToolUse)

Hook 1: 파일 저장 시 자동 포맷


```

json
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Write|Edit|MultiEdit",
        "hooks": [{
          "type": "command",
          "command": "prettier --write $CLAUDE_TOOL_INPUT_PATH 2>/dev/null; eslint -
        }
      ]
    }
  }
}

```

"command": "prettier --write \$CLAUDE_TOOL_INPUT_PATH 2>/dev/null; eslint --fix \$CLAUDE_TOOL_INPUT_PATH 2>/dev/null; exit 0"

Hook 2: Python 파일 수정 시 자동 린트

```

json
{
  "PostToolUse": [{
    "matcher": "Write|Edit",
    "hooks": [{
      "type": "command",
      "command": "bash -c 'if echo \"$CLAUDE_TOOL_INPUT_PATH\" | grep -q \"\\.py$\";
    }
  ]
}

```

"command": "bash -c 'if echo W"\$CLAUDE_TOOL_INPUT_PATHW" | grep -q W"WW.py\$W"; then ruff check --fix W"\$CLAUDE_TOOL_INPUT_PATHW" && black W"\$CLAUDE_TOOL_INPUT_PATHW"; fi; exit 0'"

Hook 3: 코드 변경 후 자동 테스트 실행

```

json
{
  "PostToolUse": [{
    "matcher": "Write|Edit",
    "hooks": [{
      "type": "command",
      "command": "npm test -- --passWithNoTests 2>&1 | tail -5",
      "async": true
    }]
  }]
}

```

Hook 4: 작업 완료 시 macOS 알림

```

json
{
  "Stop": [{
    "matcher": "",
    "hooks": [{
      "type": "command",
      "command": "osascript -e 'display notification \"Claude가 작업을 완료했습니다\" wi
    }]
  }]
}

```

"command": "osascript -e 'display notification ₩"Claude가 작업을 완료했습니다₩" with title ₩"Claude Code₩"'"

Hook 5: 세션 종료 시 자동 git 커밋

```

json
{
  "SessionEnd": [{
    "hooks": [{
      "type": "command",
      "command": "cd $CLAUDE_CWD && git diff --quiet || git add -A && git commit -m
    }]
  }]
}

```

```
"command": "cd $CLAUDE_CWD && git diff --quiet || git add -A && git commit -m 'auto: claude session $(date +%Y%m%d-%H%M)'"
```

● 보안 차단 (PreToolUse)

Hook 6: 위험 명령어 차단

```
bash

#!/bin/bash
# .claude/hooks/block-dangerous.sh
INPUT=$(cat)

COMMAND=$(echo "$INPUT" | python3 -c "
import json, sys
data = json.load(sys.stdin)
print(data.get('tool_input', {}).get('command', ''))
")

# 위험 패턴 목록
BLOCKED_PATTERNS=(
    "rm -rf /"
    "DROP TABLE"
    "DROP DATABASE"
    "git push --force"
    "chmod 777"
    "> /dev/sda"
)

for pattern in "${BLOCKED_PATTERNS[@]}; do
    if echo "$COMMAND" | grep -qi "$pattern"; then
        echo "{\"decision\": \"block\", \"reason\": \"위험 명령어 차단: $pattern\"}" >&2
        exit 2
    fi
done

exit 0
```

```
json
{
  "PreToolUse": [{
    "matcher": "Bash",
    "hooks": [{
      "type": "command",
      "command": ".claude/hooks/block-dangerous.sh"
    }]
  }]
}
```

Hook 7: .env / 시크릿 파일 보호

bash

```
#!/bin/bash
# .claude/hooks/protect-secrets.sh
INPUT=$(cat)
FILE_PATH=$(echo "$INPUT" | python3 -c "
import json, sys
data = json.load(sys.stdin)
print(data.get('tool_input', {}).get('path', ''))
")

PROTECTED=(
    ".env"
    ".env.production"
    "secrets.json"
    "*.pem"
    "*.key"
    "credentials.json"
)

for pattern in "${PROTECTED[@]"; do
    if [[ "$FILE_PATH" == *"$pattern"* ]]; then
        echo "보호된 파일 수정 차단: $FILE_PATH" >&2
        exit 2
    fi
done
exit 0
```

Hook 8: main 브랜치 직접 푸시 차단

```

bash

#!/bin/bash
INPUT=$(cat)
COMMAND=$(echo "$INPUT" | python3 -c "
import json, sys
d = json.load(sys.stdin)
print(d.get('tool_input', {}).get('command', ''))
")

CURRENT_BRANCH=$(git branch --show-current 2>/dev/null)

if echo "$COMMAND" | grep -q "git push" && [ "$CURRENT_BRANCH" = "main" ]; then
    echo "🚫 main 브랜치 직접 푸시 차단. PR을 통해 머지하세요." >&2
    exit 2
fi
exit 0

```

Hook 9: AI 판단으로 보안 검증 (Prompt Hook)

```

json
{
  "PreToolUse": [{
    "matcher": "Bash",
    "hooks": [{
      "type": "prompt",
      "prompt": "다음 bash 명령어가 안전한지 평가해줘:\n$ARGUMENTS\n\n다음  
경우에만 BLOCK을 반환해:\n- 파일 시스템 전체 삭제\n- 네트워크로 민감  
데이터 전송\n- sudo 권한 남용\n그 외는 ALLOW"
    }]
  }]
}

```

"prompt": "다음 bash 명령어가 안전한지 평가해줘:\n\$ARGUMENTS\n\n다음
경우에만 BLOCK을 반환해:\n- 파일 시스템 전체 삭제\n- 네트워크로 민감
데이터 전송\n- sudo 권한 남용\n그 외는 ALLOW"

● 컨텍스트 주입 (SessionStart)

Hook 10: 세션 시작 시 프로젝트 상태 자동 로드

bash

```
#!/bin/bash
# .claude/hooks/session-context.sh
echo "=== 세션 시작 컨텍스트 ==="
echo "현재 브랜치: $(git branch --show-current)"
echo "최근 커밋: $(git log --oneline -3)"
echo ""
echo "변경된 파일:"
git status --short | head -10
echo ""
echo "진행 중인 TODO:"
grep -r "TODO\|FIXME" src/ --include="*.py" | head -5
```

json

```
{
  "SessionStart": [{
    "matcher": "startup",
    "hooks": [{
      "type": "command",
      "command": ".claude/hooks/session-context.sh"
    }]
  }]
}
```

Hook 11: 환경 변수 자동 로드

```
bash
```

```
#!/bin/bash
```

```
# 세션 시작 시 .env.development 로드
```

```
if [ -f ".env.development" ]; then
```

```
    export $(cat .env.development | grep -v '^#' | xargs)
```

```
    echo "환경 변수 로드 완료: .env.development"
```

```
fi
```

● 품질 관리 (Stop / TaskCompleted)

Hook 12: 작업 완료 시 강제 검증

bash

```
#!/bin/bash
# .claude/hooks/enforce-completion.sh
# 테스트가 통과했는지 확인
if [ -f "package.json" ]; then
    TEST_RESULT=$(npm test -- --passWithNoTests 2>&1)
    if echo "$TEST_RESULT" | grep -q "FAIL"; then
        echo "⚠ 테스트 실패 - Claude에게 수정 요청" >&2
        echo "실패한 테스트:\n$(echo "$TEST_RESULT" | grep FAIL)" >&2
        exit 2 # Stop hook에서 exit 2 = Claude 계속 실행
    fi
fi
exit 0
```

json

```
{
  "Stop": [{
    "hooks": [{
      "type": "command",
      "command": ".claude/hooks/enforce-completion.sh"
    }]
  }]
}
```

Hook 13: 컨텍스트 압축 전 트랜스크립트 백업

bash

```
#!/bin/bash
# .claude/hooks/backup-before-compact.sh
BACKUP_DIR=".claude/backups"
mkdir -p "$BACKUP_DIR"
TIMESTAMP=$(date +%Y%m%d-%H%M%S)
cp ".claude/transcript.jsonl" \
  "$BACKUP_DIR/backup-$TIMESTAMP.jsonl" 2>/dev/null || true
echo "백업 완료: $BACKUP_DIR/backup-$TIMESTAMP.jsonl"
```

json

```
{
  "PreCompact": [{
    "hooks": [{
      "type": "command",
      "command": ".claude/hooks/backup-before-compact.sh"
    }]
  }]
}
```

● 알림 및 모니터링

Hook 14: Slack 알림 전송 (HTTP Hook)

json

```
{
  "Stop": [{
    "hooks": [{
      "type": "http",
      "url": "https://hooks.slack.com/services/YOUR/WEBHOOK/URL",
      "headers": {"Content-Type": "application/json"},
      "timeout": 5
    }]
  }]
}
```

python

```
# 서버 사이드: Hook 수신 및 Slack 전송
from flask import Flask, request
import requests

app = Flask(__name__)

@app.post("/hooks/stop")
def on_claude_stop():
    data = request.json
    requests.post(SLACK_WEBHOOK, json={
        "text": f"✅ Claude 작업 완료\n세션: {data['session_id']}"
    })
    return {"continue": True}
```

Hook 15: 작업 로그 자동 기록

```
json
{
  "PostToolUse": [{
    "matcher": ".*",
    "hooks": [{
      "type": "command",
      "command": "echo \"$(date -Iseconds) $CLAUDE_HOOK_EVENT_NAME $CLAUDE_TOOL_NAME  
$CLAUDE_TOOL_NAME\" >> .claude/audit.log",
      "async": true
    }]
  }]
}
```

"command": "echo ₩\$(date -Iseconds) \$CLAUDE_HOOK_EVENT_NAME
\$CLAUDE_TOOL_NAME₩" >> .claude/audit.log",

● 고급 패턴

Hook 16: Skills/CLAUDE.md에서 Hook 정의 (인라인) 이 Skill은 각 Bash 명령어 전에 보안 검증 스크립트를 실행하는 PreToolUse Hook을 정의합니다:

```
markdown
---
name: secure-operations
description: 보안 검사가 포함된 작업 수행
hooks:
  PreToolUse:
    - matcher: "Bash"
      hooks:
        - type: command
          command: "./scripts/security-check.sh"
---
```

Hook 17: 비동기 Hook으로 성능 최적화 async: true를 추가하면 Claude의 실행을 차단하지 않고 백그라운드에서 Hook을 실행합니다. 2026년 1월 Anthropic이 릴리즈했습니다.

```
json
{
  "PostToolUse": [{
    "matcher": "Write|Edit",
    "hooks": [{
      "type": "command",
      "command": "node heavy-analysis.js",
      "async": true,
      "timeout": 30
    }]
  }]
}
```

Hook 18: JSON Output으로 정밀 제어

```
bash
#!/bin/bash
INPUT=$(cat)
# 단순 exit code 대신 구조화된 JSON 반환
echo '{
  "continue": false,
  "stopReason": "보안 정책 위반: .env 파일 수정 시도",
  "suppressOutput": false
}'
exit 2
```

Hook 19: 파일 변경 감지 (FileChanged)

```

json
{
  "FileChanged": [{
    "matcher": "src/api/*\\.py$",
    "hooks": [{
      "type": "command",
      "command": "python -m pytest tests/test_api.py -q --tb=short",
      "async": true
    }]
  }]
}

```

Hook 20: Enterprise 조직 전체 강제 적용 Enterprise 관리자는 allowManagedHooksOnly를 사용해 사용자, 프로젝트, 플러그인 혹은 차단할 수 있습니다. 관리 설정의 enabledPlugins에서 강제 활성화된 플러그인의 Hook은 예외로, 관리자가 조직 마켓플레이스를 통해 검증된 Hook을 배포할 수 있습니다.

```

json
// 조직 관리 settings.json
{
  "allowManagedHooksOnly": true,
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [{
        "type": "command",
        "command": "/org/scripts/compliance-check.sh"
      }]
    }]
  }
}

```

8. JSON Input 스키마 (Hook이 받는 데이터)

```
json

// 모든 Hook이 stdin으로 받는 공통 JSON
{
  "session_id": "abc123",
  "cwd": "/my/project",
  "hook_event_name": "PreToolUse",

  // PreToolUse / PostToolUse 전용
  "tool_name": "Bash",
  "tool_input": {
    "command": "rm -rf node_modules"
  },

  // PostToolUse 전용 (도구 결과)
  "tool_response": {
    "output": "명령어 실행 결과..."
  },

  // Stop 전용
  "stop_hook_active": false
}
```

9. Hook 디버깅

```
bash
```

```
# 방법 1: Hook 직접 테스트
```

```
echo '{"tool_name":"Bash","tool_input":{"command":"rm -rf /"}}' \  
| ./claude/hooks/block-dangerous.sh
```

```
echo "Exit code: $?" # 기대값: 2 (차단)
```

```
# 방법 2: 디버그 모드로 Claude Code 실행
```

```
claude --debug
```

```
# 방법 3: Hook 브라우저 열기
```

```
/hooks # Claude Code 내에서 실행
```

```
# 방법 4: 감사 로그 확인
```

```
tail -f .claude/audit.log
```

10. 전체 실전 설정 예시

json

```
// .claude/settings.json (프로덕션 레벨 설정)
{
  "hooks": {

    "SessionStart": [{
      "matcher": "startup",
      "hooks": [{
        "type": "command",
        "command": ".claude/hooks/load-context.sh"
      }]
    }],

    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [{
          "type": "command",
          "command": ".claude/hooks/block-dangerous.sh"
        }]
      },
      {
        "matcher": "Write|Edit|MultiEdit",
        "hooks": [{
          "type": "command",
          "command": ".claude/hooks/protect-secrets.sh"
        }]
      }
    ],
  },
}
```

```

"PostToolUse": [
  {
    "matcher": "Write|Edit|MultiEdit",
    "hooks": [
      {
        "type": "command",
        "command": "prettier --write $CLAUDE_TOOL_INPUT_PATH 2>/dev/null; exit 0"
      },
      {
        "type": "command",
        "command": "npm test -- --passWithNoTests --silent",
        "async": true
      }
    ]
  }
],

"PreCompact": [{
  "hooks": [{
    "type": "command",
    "command": ".claude/hooks/backup-transcript.sh"
  }]
}],

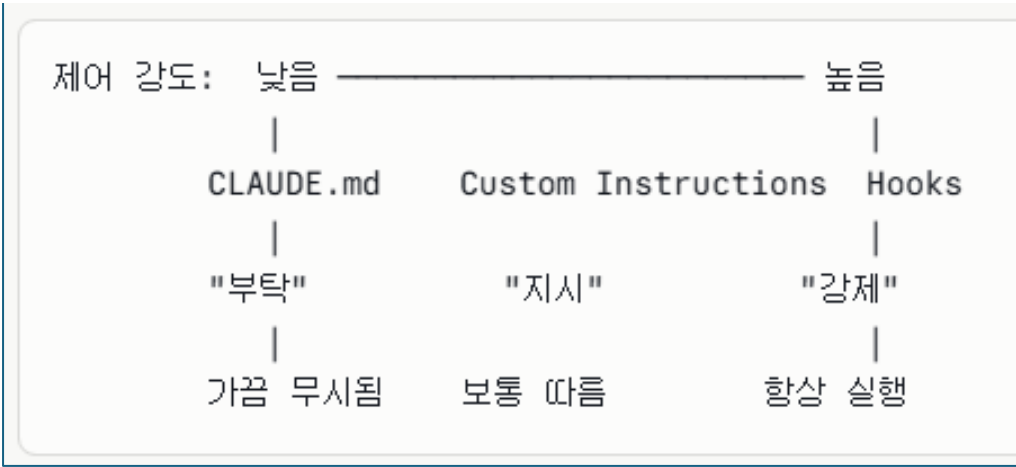
"Stop": [{
  "hooks": [
    {
      "type": "command",
      "command": "osascript -e 'display notification \"완료\" with title \"Claude Done\"'"
    },
    {
      "type": "http",
      "url": "http://localhost:8080/claude-done",
      "async": true
    }
  ]
}]
}
}

```

```
"command": "prettier --write $CLAUDE_TOOL_INPUT_PATH 2>/dev/null; exit 0"
```

[illegible]

11. Hooks vs 다른 제어 방법 최종 비교



상황	사용할 것
코딩 스타일 선호도	CLAUDE.md
응답 톤/형식	Custom Instructions
반드시 실행해야 할 포매팅	Hooks (PostToolUse)
절대 허용 안 되는 명령어	Hooks (PreToolUse, exit 2)
팀 전체 정책 강제	Hooks (Enterprise 관리)
작업 완료 알림	Hooks (Stop)

다음으로 **RAG 심화** (벡터 DB + 검색 전략), Claude Code 실전 워크플로우 통합 설계, 또는 지금까지 배운 모든 Agentic AI 개념을 통합하는 실전 시스템 구축으로 이어갈까요? 🚀

Windows 바이브 코딩 환경 완전 세팅 가이드

(스마트폰 불량·진단 데이터 분석 + 웹 대시보드 특화)

📖 전체 설치 로드맵

STEP 1: 기반 환경	STEP 2: 개발 도구	STEP 3: Claude 설정
Windows Terminal WSL2 (Ubuntu) Node.js	VS Code Python 환경 Git Docker	Claude Code CLI VS Code 확장 CLAUDE.md 설정 MCP Servers
STEP 4: 프로젝트 스택	STEP 5: 바이브 코딩	
FastAPI (백엔드) PostgreSQL + Redis React + Plotly Pandas / Scikit-learn	첫 바이브 코딩 워크플로우 설정 팀 협업 설정	

STEP 1: 기반 환경 구축

1-1. Windows Terminal 설치

Claude Code Windows 사용자에게는 Windows Terminal 앱을 추천합니다.
Microsoft Store나 GitHub 릴리즈에서 다운로드할 수 있습니다.

```
powershell

# Microsoft Store에서 설치하거나
# PowerShell (관리자)에서:
winget install Microsoft.WindowsTerminal
```

1-2. WSL2 설치 (핵심 단계)

Claude Code는 WSL2를 통한 Windows 설치를 지원합니다. Windows 11에서 WSL2 설치에 단 하나의 명령어로 가능할 만큼 간단합니다. 시작 버튼을 우클릭해 "터미널(관리자)"를 선택하세요.

```
powershell

# PowerShell (관리자 권한으로 실행)
wsl --install

# 재부팅 후 Ubuntu가 자동 시작됨
# 사용자명 + 비밀번호 설정

# 설치 확인
wsl --list --verbose
# 출력: Ubuntu Running 2 ← Version 2 확인
```

⚠ **BIOS 설정 확인:** 하드웨어 가상화가 BIOS/UEFI에서 활성화되어 있어야 합니다. 대부분 현대 머신은 이미 활성화되어 있지만 WSL2 설치가 실패하면 BIOS에서 "Intel VT-x", "Intel Virtualization Technology", 또는 "AMD-V" 설정을 확인하세요.

1-3. WSL2 메모리 최적화

```
# Windows에서 %USERPROFILE%\.wslconfig 파일 생성
# (데이터 분석용으로 넉넉하게 설정)
```

```
ini
```

```
[wsl2]
```

```
memory=12GB      # 전체 RAM의 50~60% 권장
processors=6      # CPU 코어 수
swap=4GB
localhostForwarding=true
```

1-4. Node.js 설치 (Claude Code 필수)

```
bash

# WSL2 Ubuntu 터미널에서:

# nvm 설치 (Node 버전 관리)
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.1/install.sh | bash
source ~/.bashrc

# Node.js LTS 설치
nvm install --lts
nvm use --lts

# 확인
node --version    # v22.x.x 이상
npm --version
```

STEP 2: 개발 도구 설치

2-1. Python 환경 (pyenv + uv)

pyenv와 uv를 이용한 Python 설정은 현대적이고 빠른 Python 개발 환경을 제공합니다.

```
bash

# pyenv 설치
curl https://pyenv.run | bash

# ~/.bashrc에 추가
echo 'export PYENV_ROOT="$HOME/.pyenv"' >> ~/.bashrc
echo 'command -v pyenv >/dev/null || export PATH="$PYENV_ROOT/bin:$PATH"' >> ~/.bashrc
echo 'eval "$(pyenv init -)"' >> ~/.bashrc
source ~/.bashrc

# Python 3.12 설치
pyenv install 3.12.3
pyenv global 3.12.3
python --version # Python 3.12.3

# uv 설치 (pip보다 10~100배 빠른 패키지 매니저)
curl -LsSf https://astral.sh/uv/install.sh | sh
source ~/.bashrc
```

```
echo 'command -v pyenv >/dev/null || export PATH="$PYENV_ROOT/bin:$PATH"'
>> ~/.bashrc
```

2-2. 프로젝트 전용 패키지 설치

```
bash

# 프로젝트 디렉토리 생성
mkdir ~/smartphone-analytics && cd ~/smartphone-analytics

# uv로 가상환경 생성
uv venv .venv
source .venv/bin/activate

# 데이터 분석 + 웹 백엔드 핵심 패키지
uv pip install \
    # 웹 프레임워크
    fastapi uvicorn[standard] \
    # 데이터 분석
    pandas numpy scipy scikit-learn \
    # 시각화
    plotly matplotlib seaborn \
    # DB 연결
    sqlalchemy psycopg2-binary redis \
    # 데이터 검증
    pydantic python-dotenv \
    # 유틸
    httpx pytest pytest-asyncio
```

2-3. VS Code 설치 및 WSL 연결

```
bash

# WSL2 터미널에서 VS Code 실행
# (Windows에 VS Code가 설치되어 있어야 함)
code .

# VS Code가 자동으로 WSL Remote 모드로 열림
```


VS Code 필수 확장 설치 (Ctrl+Shift+X):

필수 확장 목록:

✓ Claude Code	(Anthropic 공식, 2M+ 설치)
✓ Python	(ms-python.python)
✓ Pylance	(ms-python.vscode-pylance)
✓ Remote - WSL	(ms-vscode.remote.remote-wsl)
✓ GitLens	(eamodio.gitlens)
✓ Docker	(ms-azuretools.vscode-docker)
✓ REST Client	(humao.rest-client)
✓ Thunder Client	(rangav.vscode-thunder-client)
✓ Jupyter	(ms-toolsai.jupyter)
✓ autoDocstring	(njpwner.autodocstring)
✓ Error Lens	(usernamehw.errorlens)
✓ Prettier	(esbenp.prettier-vscode)

2-4. Git 설정

```
bash

git config --global user.name "Your Name"
git config --global user.email "your@email.com"
git config --global core.autocrlf input # WSL2 필수
git config --global init.defaultBranch main

# GitHub SSH 키 설정
ssh-keygen -t ed25519 -C "your@email.com"
cat ~/.ssh/id_ed25519.pub # GitHub에 등록
```

2-5. Docker 설치

```
bash

# WSL2에서 Docker 설치
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $USER

# Docker Compose V2
sudo apt install docker-compose-v2

# 재시작 후 확인
docker --version
docker compose version
```

STEP 3: Claude Code 설치 및 설정

3-1. Claude Code CLI 설치

Claude Code는 이제 네이티브 Windows 설치를 지원하며 Node.js가 필요 없습니다. 네이티브 인스톨러(권장)는 macOS, Windows, Linux에서 작동합니다.

```
bash

# WSL2 터미널에서:
npm install -g @anthropic-ai/claude-code

# 설치 확인
claude --version

# 첫 실행 & 로그인 (브라우저 OAuth)
claude
# → 브라우저에서 claude.ai 로그인
# → Pro/Max 플랜 필요 ($20~$200/월)
```

3-2. VS Code Claude Code 확장 설치

VS Code에서 Ctrl+Shift+X를 눌러 Extensions 뷰를 열고 "Claude Code"를 검색해 설치합니다. 확장은 Cursor, Windsurf 등 다른 VS Code 포크에서도 작동합니다.

VS Code Extensions → "Claude Code" 검색 → Anthropic 공식 설치
설치 후: 왼쪽 사이드바에 ⚡ (Spark) 아이콘 생성

3-3. 프로젝트 CLAUDE.md 설정

```
bash  
  
cd ~/smartphone-analytics  
claude /init # CLAUDE.md 자동 생성
```

프로젝트에 맞게 편집:

markdown

Smartphone Analytics 프로젝트 가이드

프로젝트 개요

스마트폰 불량·진단 데이터 분석 및 웹 대시보드 시스템

- 불량 패턴 탐지, 진단 데이터 분석, 실시간 대시보드 제공

기술 스택

- Backend: FastAPI + Python 3.12
- Database: PostgreSQL 16 + Redis (캐싱)
- Frontend: React 18 + TypeScript + Plotly/Recharts
- Data: Pandas + Scikit-learn + Scipy
- 배포: Docker + Docker Compose

프로젝트 구조

```
smartphone-analytics/  
├── backend/  
│   ├── api/           # FastAPI 라우터  
│   ├── models/        # SQLAlchemy 모델  
│   ├── services/      # 비즈니스 로직  
│   ├── analysis/      # 데이터 분석 모듈  
│   └── schemas/        # Pydantic 스키마  
├── frontend/  
│   ├── src/  
│   │   ├── components/  
│   │   ├── pages/  
│   │   └── charts/    # 대시보드 차트  
└── data/  
    ├── raw/           # 원본 진단 데이터  
    └── processed/     # 전처리 데이터
```

자주 쓰는 명령어

- 백엔드 시작: ``cd backend && uvicorn main:app --reload``
- 프론트엔드: ``cd frontend && npm run dev``
- 전체 실행: ``docker compose up -d``
- 테스트: ``pytest backend/tests/ -v``
- DB 마이그레이션: ``alembic upgrade head``

코딩 컨벤션

- Python: PEP 8, 타입 힌트 필수, docstring 필수
- 데이터 분석 함수: 입력/출력 타입, 알고리즘 설명 주석
- API 엔드포인트: RESTful, OpenAPI 문서 자동 생성
- 커밋: Conventional Commits (feat/fix/data/analysis)

도메인 지식

- 불량 유형: HW결함(배터리/화면/카메라), SW결함, 간헐적 오류
- 진단 데이터: 배터리 사이클, 온도, 충전 패턴, 센서 로그
- 핵심 지표: 불량률, MTBF, 진단 정확도, 모델별 불량 분포

절대 하지 말 것

- 고객 개인정보 포함 데이터 하드코딩 금지
- .env 파일 git 커밋 금지
- 분석 결과 검증 없이 배포 금지

3-4. .claude/settings.json Hook 설정

```

json
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Write|Edit|MultiEdit",
        "hooks": [{
          "type": "command",
          "command": "cd $CLAUDE_CWD && python -m ruff check --fix $CLAUDE_TOOL_INPU
          "async": true
        }]
      }
    ],
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [{
          "type": "command",
          "command": "bash -c 'CMD=$(echo \"$CLAUDE_TOOL_INPUT\" | python3 -c \"impo
        }]
      }
    ],
    "Stop": [{
      "hooks": [{
        "type": "command",
        "command": "echo '✅ Claude 작업 완료' | wall 2>/dev/null; exit 0",
        "async": true
      }]
    }]
  }
}

```

"command": "cd \$CLAUDE_CWD && python -m ruff check --fix
\$CLAUDE_TOOL_INPUT_PATH 2>/dev/null; exit 0",

"command": "bash -c 'CMD=\$(echo W"\$CLAUDE_TOOL_INPUTW" | python3 -c
W"import json,sys;
print(json.load(sys.stdin).get(WWW"commandWWW",WWW"WWW"))W"); echo
W"\$CMDW" | grep -qiE W"rm -rf|DROP TABLE|DROP DATABASEW" && exit 2 || exit
0"'

3-5. MCP Servers 설정 (강력 추천)

bash

~/.claude/settings.json에 MCP 설정 추가

json

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem",
        "/home/user/smartphone-analytics"]
    },
    "postgres": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-postgres"],
      "env": {
        "POSTGRES_CONNECTION_STRING": "postgresql://user:pass@localhost/analytics_db"
      }
    },
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {"GITHUB_TOKEN": "your_token"}
    }
  }
}
```

"POSTGRES_CONNECTION_STRING":

"postgresql://user:pass@localhost/analytics_db"

STEP 4: 프로젝트 스택 설치

4-1. PostgreSQL + Redis (Docker Compose)

yaml

```
# docker-compose.yml
```

```
version: '3.8'
```

```
services:
```

```
  postgres:
```

```
    image: postgres:16-alpine
```

```
    environment:
```

```
      POSTGRES_DB: analytics_db
```

```
      POSTGRES_USER: analyst
```

```
      POSTGRES_PASSWORD: ${DB_PASSWORD}
```

```
    ports:
```

```
      - "5432:5432"
```

```
    volumes:
```

```
      - postgres_data:/var/lib/postgresql/data
```

```
  redis:
```

```
    image: redis:7-alpine
```

```
    ports:
```

```
      - "6379:6379"
```

```
    volumes:
```

```
      - redis_data:/data
```

```
  pgadmin:
```

```
    image: dpage/pgadmin4
```

```
    environment:
```

```
      PGADMIN_DEFAULT_EMAIL: admin@company.com
```

```
      PGADMIN_DEFAULT_PASSWORD: admin
```

```
    ports:
```

```
      - "5050:80"
```

```
volumes:
```

```
  postgres_data:
```

```
  redis_data:
```



```
bash
```

```
docker compose up -d
```

4-2. 프론트엔드 (React + 대시보드)

```
bash
```

```
cd ~/smartphone-analytics
```

```
npx create-react-app frontend --template typescript
```

```
cd frontend
```

```
# 대시보드 필수 패키지
```

```
npm install \
  plotly.js react-plotly.js \
  recharts \
  @tanstack/react-query \
  axios \
  tailwindcss \
  @headlessui/react \
  date-fns \
  react-router-dom
```

4-3. 백엔드 프로젝트 구조 생성

```
bash

mkdir -p backend/{api,models,services,analysis,schemas,tests}

# .env 파일 생성
cat > .env << EOF
DATABASE_URL=postgresql://analyst:password@localhost/analytics_db
REDIS_URL=redis://localhost:6379
SECRET_KEY=$(openssl rand -hex 32)
ENVIRONMENT=development
EOF
```

STEP 5: 바이브 코딩 시작

5-1. 첫 바이브 코딩 실전 예시

VS Code에서 ⚡ 아이콘 클릭 후:

"스마트폰 진단 데이터 분석 시스템의 기초를 만들어줘."

요구사항:

1. FastAPI 백엔드 - 진단 데이터 수신 API
2. PostgreSQL 모델 - 기기정보, 불량기록, 진단로그 테이블
3. 기본 불량 패턴 분석 함수 (배터리/화면/카메라별)
4. React 대시보드 - 불량률 추이 차트 (Plotly)

CLAUDE.md의 프로젝트 구조를 따라줘."

5-2. 단계별 바이브 코딩 프롬프트 모음

데이터 모델링:

"스마트폰 진단 데이터를 위한 SQLAlchemy 모델을 만들어줘.
포함할 내용:

- Device: 기기 시리얼, 모델명, 제조일, 판매일
- DiagnosticLog: 진단 시간, 배터리 사이클, 온도, 센서값들
- DefectRecord: 불량 유형, 발견일, 심각도, 처리상태
- 모든 모델에 created_at, updated_at 자동 처리"

분석 모듈:

"스마트폰 불량 패턴 탐지 분석 모듈을 만들어줘.

- 모델별 불량률 계산
- 시계열 불량 트렌드 분석 (rolling window)
- 이상치 탐지 (IQR 방식)
- 배터리 성능 저하 예측 (선형 회귀)
- 결과는 dict로 반환, 모든 함수에 타입 힌트 포함"

대시보드:

"React + Plotly로 불량 분석 대시보드를 만들어줘.

- 상단: KPI 카드 (총 기기수, 불량률, 평균 수명)
- 좌측: 모델별 불량률 바 차트
- 우측: 월별 불량 트렌드 라인 차트
- 하단: 불량 유형별 파이 차트 + 최근 불량 테이블
- Tailwind CSS로 스타일링, 다크모드 지원"

5-3. 권장 바이브 코딩 워크플로우

① 아침 세션 시작:

`/clear`

"어제 진행한 불량 분석 API 이어서 작업해줘.
현재 상태는 `CLAUDE.md` 참조."

② 새 기능 추가:

`/plan`

"배터리 수명 예측 기능을 추가하고 싶어.
구현 계획을 먼저 보여줘."

→ 계획 승인 후 → 실제 구현

③ 막혔을 때:

"이 에러 메시지 분석해줘:

[에러 붙여넣기]

관련 파일: `backend/analysis/battery.py`"

④ 코드 리뷰:

`/security-review`

"방금 만든 API 엔드포인트 보안 검토해줘"

⑤ 작업 완료:

"오늘 작업 내용 요약해서 `CLAUDE.md` 업데이트해줘"

전체 설치 체크리스트

기반 환경:

- ☐ Windows Terminal 설치
- ☐ WSL2 Ubuntu 설치 및 메모리 설정
- ☐ Node.js (nvm) 설치 - v22 이상
- ☐ Git 설치 및 설정

Python 환경:

- ☐ pyenv 설치
- ☐ Python 3.12 설치
- ☐ uv 패키지 매니저 설치
- ☐ 프로젝트 가상환경 생성
- ☐ 분석/웹 패키지 설치

VS Code:

- ☐ VS Code 최신 버전 (1.98+)
- ☐ Remote WSL 확장
- ☐ Claude Code 확장 (Anthropic 공식)
- ☐ Python, Pylance 확장
- ☐ GitLens, Docker 확장
- ☐ Jupyter 확장

Claude Code:

- ☐ CLI 설치 및 로그인
- ☐ Pro/Max 플랜 구독
- ☐ 프로젝트 CLAUDE.md 작성
- ☐ .claude/settings.json Hooks 설정
- ☐ MCP Servers 설정 (PostgreSQL, GitHub)

프로젝트 스택:

- ☐ Docker + Docker Compose
- ☐ PostgreSQL + Redis 컨테이너 실행
- ☐ FastAPI 백엔드 기초 구조
- ☐ React + TypeScript 프론트엔드
- ☐ Plotly/Recharts 대시보드 패키지

요금:

- ☐ Claude Pro (\$20/월) - 일반 사용
- ☐ Claude Max (\$100/월) - 집중 개발 시 권장

최종 아키텍처 그림



이 환경이 구축되면 "스마트폰 모델별 불량률 대시보드 만들어줘"처럼 자연어로 말하면 Claude가 분석 코드부터 대시보드까지 완성해주는 진정한 바이브 코딩이 가능합니다. 특정 단계에서 막히는 부분이 있으면 바로 알려주세요! 🚀